VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Throughput and RTT Analysis of IPv4 and IPv6 in Dual-Stack Environments Using GNS3

By

Lê Đăng Khoa

ITITIU21227

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Information Technology/Computer Science/Computer Engineering

Ho Chi Minh City, Vietnam
2026

Throughput and RTT Analysis of IPv4 and IPv6 in Dual-Stack Environments Using GNS3

APPROVED BY:

_____ ,

Le Hai Duong, Ph.D

THESIS COMMITTEE

ACKNOWLEGMENTS

I would like to thank all of you deeply who have assisted me in the preparation process of this special study report. To begin with, I would like to congratulate the brilliant advice of my advisor, Prof. Le Hai Duong, their detailed comments, and steady support. Their knowledge in computer networking and internet protocol has largely influenced the path and the level of this study.

I am profoundly grateful to my family and friends for their moral support, patience, and motivation, which have been indispensable during the course of this research.

TABLE OF CONTENTS

ACKNOWLEGMENTS **3**

TABLE OF CONTENTS **4**

LIST OF FIGURES **7**

LIST OF TABLES **9**

ABSTRACT **10**

CHAPTER 1 ..... **11**

INTRODUCTION **11**

1.1. Background 11

1.2. Problem Statement 12

1.3. Scope and Objectives 13

1.4. Scope and Limitation 14

CHAPTER 2 ..... **15**

LITERATURE REVIEW **15**

2.1. IPv6 Architecture and Protocol Characteristics 15

2.2. Architectural Dissimilarities between IPv4 and IPv6 16

2.3. Empirical Comparisons of IPv4 and IPv6 Performance 18

2.4. IPv6 Deployment Trends and Network Stability 18

2.5. Routing Behaviour and its effect on delay 19

2.6. TCP Behavior and Its Impact on Performance Metrics 20

2.7. Factors that affect the throughput in IPv4 and IPv6 20

2.8. IPv4 and IPv6 Factors that affect Round-trip time 21

2.9. Behaviour of TCP and Packet-Level Events 22

2.10. Sensitivity of Application protocols 22

2.11. Related work 23

CHAPTER 3 ..... **25**

METHODOLOGY **25**

3.1. Research Design 25

3.2. Network Topology 25

3.3. Protocol detail 26

3.3.1. File Transfer Protocol..... 26
3.3.2. Server Message Block.....27
3.4. Hardware and Software Setup.....27
3.4.1. Virtualization Environment.....27
3.4.2. Device Configurations..... 30
3.4.3. Device Configurations..... 31
3.5. Test Procedures..... 32
3.5.1. General Test Steps.....32
3.5.2. iPerf3 Test Procedure..... 33
3.5.3. FTP Test Procedure..... 33
3.5.4. SMB (Samba) Test Procedure..... 34
3.5.5. Wireshark Analysis Procedure..... 35
3.6. Throughput and Round-Trip time calculation.....37
3.6.1. Average throughput calculation..... 37
3.6.2. Average round-trip time (RTT) calculation.....39
CHAPTER 4..... 40
RESULT AND ANALYSIS..... 40
4.1. LAN topology..... 40
4.1.1. iPerf3..... 40
4.1.2. FTP (VSFTPD)..... 43
4.1.3. SMB (Samba).....46
4.2. WAN topology..... 50
4.2.1. iPerf3..... 50
4.2.2. FTP (VSFTPD)..... 53
4.2.3. SMB (Samba).....56
4.3. Result summary.....59
CHAPTER 5..... 62
DISCUSSION AND RECOMMENDATION..... 62
5.1. Improvement recommendation..... 62
5.1.1. Favouring Native IPv6 connectivity..... 62

5.1.2. Routing Symmetry and Traffic Engineering.....62
5.1.3. Hardware Capability / Software Stack Maturity.....63
5.1.4. Correct DNS Configuration and Name Resolution Optimization..... 63
5.1.5. Firewall, Security Policy, and Software Readiness.....63
5.1.6. Ongoing Observation and Performance Checking.....64
5.1.7. Optimization of Applications.....64
5.2. OSPFv3 cost implementation and optimization.....64
5.2.1. Multipath routing and its application in real-life..... 64
5.2.2. OSPFv3 Overview in IPv6 routing..... 66
5.2.3. Cost Metric and Path Selection Mechanism..... 66
5.2.4. Equal-Cost Multipath (ECMP) and Its Implications.....66
5.2.5. Cost-Based Routing Optimization Strategy.....67
5.2.6. Throughput result comparison..... 68
5.3. Path MTU mismatch and Path MTU Discovery.....70
5.3.1. Maximum Transfer Unit and Path MTU.....70
5.3.2. Path MTU mismatch..... 71
5.3.3. Path MTU Discovery in action..... 72
CHAPTER 6.....73
DISCUSSION, CONCLUSION AND FUTURE WORK.....73
6.1. Discussion..... 73
6.2. Conclusion..... 75
6.3. Future work.....75
REFERENCES.....76

LIST OF FIGURES

Figure 2.1: IPv4 header fields....16
Figure 2.2: IPv6 header fields....17
Figure 2.3: Test result summary of Cisco white paper....24
Figure 3.1: GNS3 network topology....26
Figure 3.2: GNS3 VM configuration....28
Figure 3.3: QEMU VM configuration....29
Figure 3.4: VM1 network configuration....31
Figure 3.5: iPerf3 client testing....33
Figure 3.6: vsftpd client authentication....33
Figure 3.7: vsftpd client retrieve data....34
Figure 3.8: smb client retrieve data....35
Figure 3.9: Wireshark I/O graph....36
Figure 3.10 & 3.11: Wireshark RTT and throughput graph....37
Figure 3.12: Python code to calculate average throughput....38
Figure 3.13: Python Code to extract the average Round-trip time....39
Figure 4.1: I/O graph of iPerf3 LAN capture during the 3rd run....40
Figure 4.2: Throughput graph of iPerf3 LAN capture during the 3rd run....41
Figure 4.3: RTT graph of iPerf3 LAN capture during the 3rd run....42
Figure 4.4: I/O graph of FTP LAN capture during the 2nd run....43
Figure 4.5: Throughput graph of FTP LAN capture during the 2nd run....44
Figure 4.6: RTT graph of FTP LAN capture during the 2nd run....45
Figure 4.7: I/O graph of SMB LAN capture during the 1st run....46
Figure 4.8: Throughput graph of SMB LAN capture during the 1st run....48
Figure 4.9: RTT graph of SMB LAN capture during the 1st run....49
Figure 4.10: I/O graph of iPerf3 WAN capture during the 1st run....50
Figure 4.11: Throughput graph of iPerf3 WAN capture during the 1st run....51
Figure 4.12: RTT graph of iPerf3 capture during the 1st run....52

Figure 4.13: I/O graph of FTP WAN capture during the 2nd run..... 53

Figure 4.14: Throughput graph of FTP WAN capture during the 2nd run.....54

Figure 4.15: RTT graph of FTP capture during the 2nd run..... 55

Figure 4.16: I/O graph of SMB WAN capture during the 3rd run..... 56

Figure 4.17: Throughput graph of FTP WAN capture during the 3rd run..... 57

Figure 4.18: RTT graph of SMB capture during the 3rd run..... 58

Figure 5.1: Multipath routing illustration.....65

Figure 5.2: OSPFv3 IPv6 experimenting network topology.....67

Figure 5.3: Throughput performance of R2 - R5 vs. R3 - R4 - R5 routes in iPerf3..... 68

Figure 5.4: Throughput performance of R2 - R5 vs. R3 - R4 - R5 routes in FTP..... 69

Figure 5.5: Throughput performance of R2 - R5 vs. R3 - R4 - R5 routes in SMB.....70

Figure 5.6: ICMPv6 “Packet too big” messages..... 72

Figure 5.7: PMTUD adjust the MTU of sender..... 73

LIST OF TABLES

Table 1: IPv4 vs IPv6 comparison.....15

Table 2: Performance measurement tools.....31

Table 3: Capture result summary.....59

ABSTRACT

The depletion of IPv4 address space has hastened the move towards implementing IPv6 all over the world but issues based on the performance of IPv6 are yet to impact the choice to adopt. In spite of the increased address space and a reduced header format addressed by and designed by IPv6, the previous literature thus presents conflicting evidence on comparison in IPv6 performance against IPv4, especially on throughput and latency. This ambiguity underlines the necessity of the restricted empirical assessment in a variety of network settings and loads of applications.

In this thesis, the performance of IPv4 and IPv6 is compared, in the Local Area Network (LAN) and Wide Area Network (WAN) setting and condition. The experiments were performed in a controlled virtualized computing setting in dual-stack configurations, and three typical applications that are commonly used: iPerf3, FTP and SMB, simulated bulk data transfer and interactive traffic patterns. Performance measurements based on throughput and round-trip time (RTT) with the help of packet analysis on TCP behavior using Wireshark.

The findings indicate that IPv6 performance is very situational and similar to IPv4 in all settings used. IPv4 had shown a small throughput improvement in bulk-transfer workloads in LAN environments, mainly because the protocol overhead was reduced, but the difference was minor and not generally significant. With interactive workloads, IPv6 had an equivalent or higher throughput stability and the stability in RTT. IPv6 has also been observed to have reduced average RTT and a less jagged distribution of delays in the WAN environment, which helps establish more stable throughput when using sustained TCP flows. There was no systematic drop in performance that could be attributed to IPv6.

Generally, the results prove IPv6 to be at least as efficient as IPv4 and potentially even better depending on the conditions on the WAN when it is introduced in a native environment and properly configured. The research confirms the inference that the adoption of IPv6 does not imply any performance penalty and affirm the research as being suitable as a long-term substitute IPv4 in contemporary networks.

CHAPTER 1

INTRODUCTION

1.1. Background

The intensive growth of the Internet during the last decades has compounded the depletion of IPv4 addresses, which is predicated on the 32-bit addressing space of the protocol which allows assigning unique addresses only to a range of about 4.3 billion [1]. This address space has been shown to be inadequate with the development of mobile devices, cloud platforms, the deployment of IoT, and the constant availability of connectivity. To address these limitations, IPv6 was built with an extremely extended 128-bit address space, offering an practically limitless address space and an almost scalable and structured allocation scheme [2]. In addition to providing increased address space, IPv6 adds a simple fixed format header that is designed to enable a router to process more quickly and forwarding to work better. It also goes towards a more clean architectural philosophy by minimizing the use of network address translation (NAT) and promotes hierarchical aggregation of prefixes and includes forward-looking mechanisms that are designed to scale the Internet in the long term.

Nevertheless, the actual performance of IPv6 has been controversial regardless of the improvement in its architecture. A significant amount of measurement literature has revealed that IPv6 behaviour is broadly divergent with maturity of deployment, and the quality of the routing environment. The initial measurements and research of transitional networks indicated an increase in latency, loss or varying stable routing paths of the IPv6, compared with IPv4 [6][7]. This was commonly due to ineffective routing infrastructure, reduced number of peering relations or use of tunneling systems that added extra overhead. The more recent studies however portray a different picture. Experiments on native, well-developed IPv6 infrastructures showed that IPv6 can match IPv4 and, in some bases, be better than the latter with regard to delay as well as connection setup period [3][4][5][8][12]. These gains reflect the continued optimization of IPv6 network architecture, operator skills and stability of backbone deployments, through long-term research on Internet-scale deployments [9][10][11].

With this conflicting evidence, and in light of the fact that the modern Internet is predominantly a two stack environment wherein both protocols are present, it is necessary to know how the two protocols perform in the real world of operation. Most organisations and service providers are still unwilling to fully depend on IPv6 by having performance concerns and this should be the case even as more organisations globally adopt IPv6. Throughput and round-trip time (RTT) measurements are of specific interest, as they have a direct impact on user experience, TCP efficiency, and the performance of typical applications. This study is valuable to this continuing debate since it provides controlled LAN and WAN experiments with various workloads iPerf3, FTP and SMB to assess whether IPv6 has performance differences compared to IPv4 under the same circumstances. These differences are important not just in explaining the current working attributes of IPv6, but also in the explanation of deployment plans as the Internet gradually evolves to IPv6.

1.2. Problem Statement

Even though IPv6 has been generally seen as the successor to the IPv4 in the long run, the issue of its actual performance is still questionable and is still affecting the decision in the implementation of IPv6 in enterprise and service-provider settings. Measurement studies have not yielded consistent results regardless of the 20 years of continued development. Other studies have found that IPv6 is associated with either delay, higher packet loss or less efficient routing in a network that is either immature or that which has been set with inconsistent traffic engineering policies when used in immature networks [6][7]. These deficiencies can frequently be related to transitional networks, distinct routing topologies or asymmetric return paths, and these may impair performance regardless of the design of the protocol.

Conversely, a number of empirical investigations indicate that there are situations when IPv6 is as good, or even superior, than IPv4, especially in those networks that support native dual-stack routing, effective prefix aggregation, and consistent peering setups [8][9][12]. The streamlined IPv6 header, simplified forwarding behaviour, and no IPv4-related legacy mechanisms may result in more predictable routes, or even lower latency routes, in such environments. This deviation in empirical results seems to imply that the protocol-level discrepancies are not sufficient to define the performance, but the routing design, equipment support, and deployment maturity are of a crucial importance. Since the factors differ substantially across networks, there is a conflict in the conclusions of the available literature

that makes the organizations unable to predict the IPv6 performance reliably in their operational networks.

The engineers and decision-makers are therefore unable to reliably obtain practical data on the behaviour of the IPv4 and IPv6 in controlled conditions, based on which the data is independent of external variables. This ambiguity makes it difficult to plan the long-term IPv6 adoption and the transition may be postponed. In an attempt to fill this gap, the current study will focus on IPv4 and IPv6 performance in closely managed LAN and WAN conditions with the same test loads. Through the measurement of throughput, RTT and TCP-level behaviour of the two protocols, the study will help to determine which protocol outperforms, underperforms, or equals in all cases, and further explains under what conditions such differences occur. This method of controlled execution assists in isolating protocol-level performance properties and generates evidence that could be utilized to embrace more effective IPv6 deployment schemes.

1.3. Scope and Objectives

The objective of this study is as follows:

- Compare IPv4 and IPv6 throughput across LAN and WAN.
- Measuring the round-trip time (RTT) of each protocol and the effect of these protocols on throughput.
- Test the performance of IPv6 against IPv4 when put through identical test conditions and determine if IPv6 is worse, the same, or better.
- Consistently track down technical causes of known performance variations, with reference to the experience of the underlying IPv6 performance research.
- Recommend effective optimizations to make IPv6 work well in actual deployments.

1.4. Scope and Limitation

This study is done in a controlled virtual environment with two VM nodes and a server. The three applications that are used in performance tests iPerf3, FTP and SMB are executed over

LAN and WAN paths. Two important measurements are considered in terms of analysis: the throughput and the round-trip time, which are standard in the literature of IPv6 measurements [6][7][8][12]. This research does not consider other factors that may introduce uncertainty in long-haul routing, or those that could affect it at the ISP level.

CHAPTER 2

LITERATURE REVIEW

2.1. IPv6 Architecture and Protocol Characteristics

The IPv6 was developed to overcome the weaknesses of the IPv4, especially those related to 32-bit address space depletion, and facilitate long-term Internet expansion. The fixed-sized base header in the standardized format of the header structure as per RFC 8200 makes routing easier and minimizes variations in processing the IPv4 header because it is variable-length [2]. This simplified format, according to academic literature, has the benefit of making per-hop parsing less complex and can affect forwarding performance in some hardware and software designs.

Table 1: IPv4 vs IPv6 comparison

IPv4	IPv6
IPv4 has a 32-bit address length.	IPv6 has a 128-bit address length.
It can generate 4.29×10^{9} address space.	The address space of IPv6 is quite large; it can produce 3.4×10^{38} address space.
Address representation of IPv4 is in decimal.	Address Representation of IPv6 is in hexadecimal.
In IPv4 checksum field is available.	In IPv6 checksum field is not available.
IPv4 has a header of 20-60 bytes.	IPv6 has a header of 40 bytes fixed.

It is also pointed out by research that variations in fragmentation behavior, neighbor discovery, and extension header can influence performance in certain situations. An example is where IPv6 moves fragmentation to the end hosts as opposed to routers, which can decrease router load but might change overheads in the processing of packets in some traffic patterns. Such differences in architecture are the basic foundation of performance comparison in subsequent empirical research.

2.2. Architectural Dissimilarities between IPv4 and IPv6

The varying features between IPv4 and IPv6 start at the very basic level; the packet header. IPv4, shown in Figure 2.1, is based on a variable-length structure (between 20 and 60 bytes) of header, with optional fields, information on fragmentation, and a checksum of the header. Each router in the route will need to recheck and recalculate this checksum and this will add processing load. Moreover, optional fields make parsing more difficult, so routers (particularly older or software-based routers) use more cycles to decide what to do with each packet. Such variability of the length of header can affect forwarding consistency in that varying routers can treat optional fields differently, based on hardware acceleration support and internal pipeline implementation.

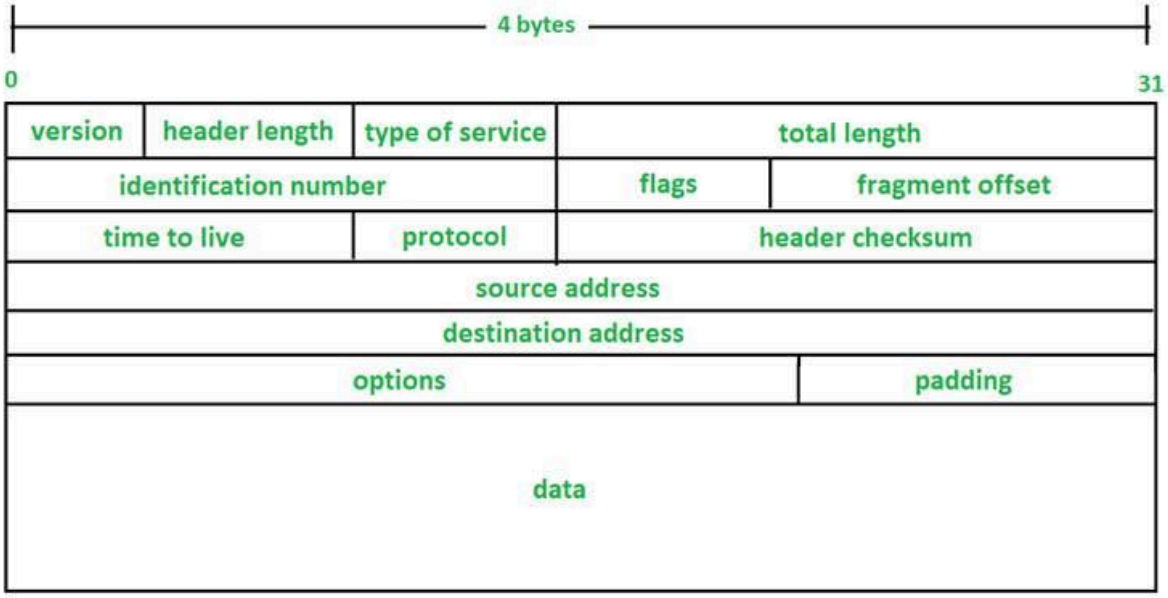


Figure 2.1: *IPv4 header fields*

As defined in RFC 8200, IPv6 simplifies this task by having a set base header, shown in Figure 2.2, that is 40 bytes and with a clean and predictable structure [2]. The IPv6 enables routers to implement forwarding decisions more quickly and simply by disabling the header checksum and relocating optional functions to modular extension header. There is also the removal of router-based fragmentation, which also helps reduce processing needs as IPv6 does not mandate routers but instead end hosts to control fragmentation. Theoretically, these enhancements enable the IPv6 packets to incur less per-hop processing time, particularly when using multi-hop routes where the repetitive computation of checksums in IPv4 sum up to recordable delay. Nevertheless, this theoretical leadership might not necessarily reflect in real measurements, as the real performance is based on routing maturity, device support, and the actual reality of IPv6 implementation.

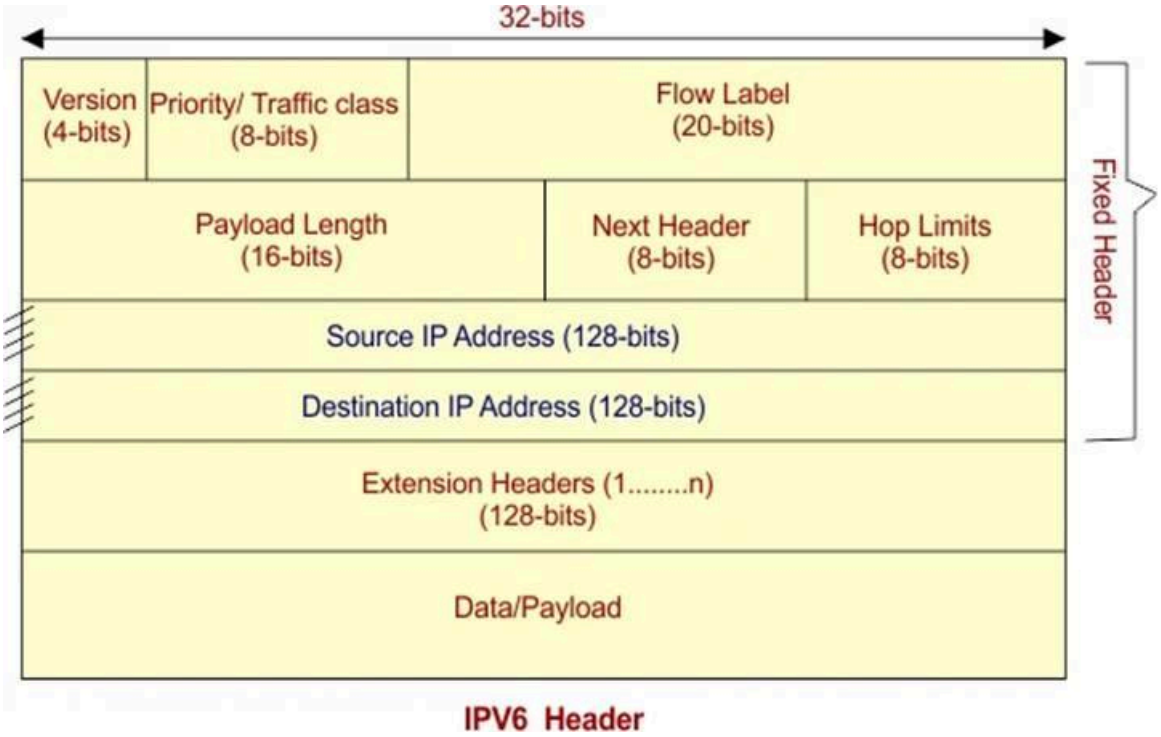


Figure 2.2: *IPv6 header fields*

The other architectural difference is on the issue of addressing. The 32-bit address space of IPv4 places a significant burden on NAT usage and introduces extra processing steps at the network boundaries and introduces some small delays. The large 128-bit address space of IPv6 in most scenarios avoids NAT and provides hierarchical addressing space which can ultimately reduce routing tables and enhance aggregation. Although these properties do not directly affect the delay or throughput at the packet level, they have an effect on network

architecture not directly on the network but indirectly on the performance when networks transition to IPv6, as opposed to transitional mechanisms.

2.3. Empirical Comparisons of IPv4 and IPv6 Performance

Much research has been conducted concerning the IPv4 and IPv6 performance over a wide range of networks. Initial measurement reports like those by Wang et al. [6] noted that distortions of IPv6 showed an increased latency in certain cases because immature routing infrastructure and route length was highest at the initial phases of deployment. This was later supported by other studies such as Zhou et al. [7] which reported quantifiable variation in delay and loss in IPv6 transit networks especially in transitional deployment periods.

On the other hand, a number of more recent studies show that under well-developed and well-optimized conditions, IPv6 can be equal or even faster than IPv4. A sequence of large-scale analyses of IPv6 traffic by Li et al. demonstrated that routing, configuration management and prefix assignment strategies with substantial effects in IPv6 performance [3][4][5]. Their results suggest that IPv6 performance is also better when the networks are run using IPv6 architecture as opposed to the use of the transitional mechanisms.

Likewise, Li et al. [8] have quantified the delay and loss as well as reordering of packets related to native IPv6 networks and concluded that IPv6 functionality is highly competitive to IPv4 when routing paths are well established and the intermediate devices are fully supported to permit IPv6. Bajpai and Schonwalder [12] also demonstrated that IPv6 also creates connections much faster than IPv4, especially when IPv6 paths are shorter or better configured. All these studies serve to show that the performance of IPv6 is highly reliant on the degree of deployment maturity, routing symmetry and hardware/software implementations.

2.4. IPv6 Deployment Trends and Network Stability

In addition to the raw performance metrics, it has also studied IPv6 networks behavior of scale. Dhamdhere et al. [9] examined the IPv6 implementation patterns in the world backbone networks and there has been an increase in the efficiency of routing and the adoption rate of IPv6 with time. Czyz et al. [10] also measured IPv6 adoption using the Internet-scale datasets and were able to discover continuous growth, but deployment is not even across regions.

Surveys also reveal that IPv6 stability of the network is also improving due to increased providers implementing native IPv6 systems. Livadariu et al. [11] have studied both control-plane and data-plane stability in a number of operating IPv6 networks and have found a trend to lower routing inconsistencies and enhanced performance parity with IPv4. Wu et al., who studied a national IPv6 backbone (CERNET2) also showed that large IPv6 deployments can attain high throughput, predictable delay properties and robust routing behavior when these are carefully designed.

These results highlight the relevance of deployment architecture and provide an indicator that performance variability in immature, IPv6-based networks might not be indicative of more sophisticated networks.

2.5. Routing Behaviour and its effect on delay

The logical routes of the two protocols may be different even though both IPv4 and IPv6 can exist in the same physical infrastructure. Most networks have IPv4 and IPv6 routing as distinct control planes, whose BGP relationships, route preference, and traffic engineering policies are different. An example of this is that IPv4 paths can be more optimised because decades of peering agreements and carefully constructed routing trees have been established whereas IPv6 paths can be in the early stages of maturity resulting in situations where IPv6 traffic can pass through longer or less efficient paths. Research by Wang et al. [6] and Zhou et al. [7] has noted that initial deployments of IPv6 tended to have more latency because of this reason. On the other hand, more recent investigations by Li and others [3][4][5] have demonstrated that routing paths in IPv6 networks tend to be the same or even better than their IPv4 counterparts as the network maturity and configuration neared perfection.

The asymmetry of routing may also be critical. Due to the possible difference in the forward or reverse paths of IPv4 and IPv6, the effective RTT of those can vary even within a single network. Such discrepancies could be as a result of various policies on load balancing, varying link capacities or various regional peering strategies. Routing is also a factor of stability. Studies of Dhamdhere et al. [9], Czyz et al. [10], and Livadariu et al. [11] show that the control-plane stability of IPv6 has grown significantly over the years, although it is still inconsistent among providers. Delay is influenced by stability, since route flaps, inconsistent prefix space advertisements, and transient convergence phenomena can temporarily raise RTT,

or bring about bursty latency behavior. Therefore, the routing layer provides a dynamic and unpredictable effect on IPv4 and IPv6 performance, especially in the WAN configurations when more than two autonomous systems, queuing points and routing policies are involved.

2.6. TCP Behavior and Its Impact on Performance Metrics

Since the paper involves the analysis of packet-level protocols like retransmissions, duplicate acknowledgments, and out-of-order packets, the literature on TCP behavior is needed. The supporting literature on the reordering of packets proves that reordering is frequent on the current networks and does not necessarily presuppose failure. As demonstrated by Laor and Gendel [13], Wang et al. [14], and Zhou and Van Mieghem [15], there is a direct effect of the throughput by the reordering of packets on the growth of the congestion window and the generation of unnecessary retransmissions.

There are other works that examine TCP behavior in case of persistent reordering or loss. TCP-PR was proposed by Bohacek et al. [16] to deal with reordering in a better way, which underlines the limitations of the traditional TCP mechanisms. Dharmapurikar and Paxson [18], Varghese et al. [19] and Vutukuru et al. [20] investigated TCP resiliency in the context of manipulating packets or significant reordering, demonstrating how the recovery logic in TCP can affect end-to-end throughput and delay.

These guidelines can be directly applied to this thesis since under IPv4 and IPv6 the indicators of protocol behavior are retransmissions, duplicate ACKs, out of order packets and window related events. Knowledge about these processes can be used to understand the variation in the empirical findings that come afterward in the research.

2.7. Factors that affect the throughput in IPv4 and IPv6

TCP-based applications, including iPerf3, FTP, and SMB, are primarily limited by the behaviour of the congestion control algorithm and not just the speed of the link. TCP tries to gradually expand its congestion window based on the promptness of acknowledgement messages received by the receiver. The rate of the congestion window expansion is directly related to the RTT and so slight variations in the latency may affect the throughput. Provided that IPv6 has slightly reduced RTT due to a faster forwarding or reduced processing overhead, it can possibly enable the congestion window to increase faster. On the other hand, in case the

IPv6 routes are longer or more volatile, then the larger RTT could reduce this increase and lead to reduced cumulative throughput.

TCP also responds very well to loss of packets. A loss of even one packet can reduce the congestion window by half and initiate a slow recovery. Researchers like Wang et al. [14] and Zhou and Van Mieghem [15] have demonstrated that reordering of packets as is often the case in multipath routing situations can also be interpreted as loss leading to unwarranted retransmission and lowered throughput. Laor and Gendel [13] also proved that slight variations in reordering patterns may lead to quantifiable variations in application throughput. These ideas play a significant role in the comparison of IPv4 and IPv6 since various routing routes or queueing sequences can alter timing relationships among packets, which indirectly influences throughput.

Lastly, TCP behaviour is non-pure under IPv4 and IPv6 because of the detail of implementation in the operating system or in network equipment. An example is that some network interface cards, firewalls, or load balancers have higher IPv4 throughput than IPv6, particularly on older hardware. On the other hand, the current systems can be more efficient in processing IPv6 due to its less complex format of the header. These implementation-specific differences may have a manifestation in slight differences in window growth, recovery timing, or even in pacing of packets each of which would lead to the ultimate throughput realization.

2.8. IPv4 and IPv6 Factors that affect Round-trip time

RTT is dependent on physical distance, hops, queue times and router processing time. Although the streamlined header of IPv6 has the potential to ensure less processing delay, empirical evidence is heavily reliant on the success of the IPv6 path establishment within the network. An example is that IPv6 can follow a more optimal path in a network with IPv6-first principles but in a network where IPv6 is a secondary protocol, it may follow a longer or less-optimal path. Additional latency may also be caused by transitional mechanisms, including tunnels or backbone sections that do not fully support IPv6.

RTT is also influenced by queueing behaviour. Although both IPv4 and IPv6 may be configured to be served by the same physical connection, they may have different forwarding queues, dissimilar load distributions, or different timing behaviours which lead to different queueing delays of their packets. This is especially applicable in the WAN path when the

bottleneck links build up bursts of packets. Applications with high frequency of small packets like SMB enhance these timing variations since every request-response sequence requires stable RTT. By comparison, FTP and iPerf3 are very dependent on sustained bulk transfer and are less sensitive to the short term variations of RTT, but very sensitive to longer term latency trends.

2.9. Behaviour of TCP and Packet-Level Events

The behaviour of TCP in various conditions of the network offers crucial information on the general performance. Retransmissions are the extreme opposite of lost packet and transmitted packets, the sender of a packet is convinced that the packet has been lost, and thus makes the congestion window to reduce significantly. Receive acknowledgments that are duplicated indicate to the sender that the packets are received in order and fast retransmit is made in most instances. Out-of-order delivery can come either by load-balanced routes or by small queueing variations, and is not necessarily an indication of congestion. A research study by Bennett et al. and Przybylski et al. [17] has revealed that reordering is typical in most backbone connections and is to be anticipated in parallel forwarding architecture.

Nevertheless, the traditional loss-recovery protocols of TCP do not handle persistent reordering very well, and researchers have suggested more robust protocols, including those of Bohacek et al. [16], Dharmapurikar and Paxson [18], Varghese et al. [19], and Vutukuru et al. [20]. These results are directly applicable to this thesis since the variations in the ordering of packets and timing anomalies and other congestion indicators might manifest differently in IPv4 and IPv6 paths related to routing maturity and infrastructure disparities. This theoretical background helps to understand why the empirical findings can indicate a difference in the behaviour of TCP which cannot be detected easily but that can be measured in the two protocols.

2.10. Sensitivity of Application protocols

The difference in IPv4 and IPv6 can be affected by which application is producing traffic in such a way that iPerf3 is used to generate sequential and high-volume TCP traffic aimed at saturating the connection, and should thus be used to evaluate maximum throughput and responsiveness to RTT or window collapse. FTP also produces bulk data transfer but has a distinct control channel of commands, which adds extra timing dependencies that can be

somewhat more sensitive to path delay or jitter. In comparison, SMB is very interactive, and a small exchange of request-response packets occurs continuously. It is highly reliant on RTT stability, consistency in the timing of packets and queueing conditions. Such workload differences justify the existence of differences in the behavior of IPv4 and IPv6 based on the application under testing.

2.11. Related work

Comparative studies of the performance of IPv4 and IPv6 have been conducted both in academia and industry with both positive and negative results depending on hardware, configuration and traffic. Another interesting industrial experiment was that by the Government Systems Engineering (GSE) team of Cisco, who carried across a variety of router platforms; those tested included the Cisco 1841, 2811, 3825, 7200, 7301 and 7606 models, to establish the possibility of IPv6 enabling causing any measurable performance overhead [21]. Their testing was driven by anxiety on the part of the large U.S federal agencies that IPv6 larger header would decrease forwarding effectiveness on routers still planned around 32-bit or 64-bit CPU look-up structures. In their measurements of throughput, latency, frame loss and CPU utilisation using Spirent TestCenter, the Cisco study discovered that IPv4 and IPv6 typically performed well in most hardware platforms, and traffic loads with only minor variations in IPv6 throughput (at very small packet sizes, say 256 bytes) on lower end software-switching models where IPv6 throughput decreased proportionally with the amount of IPv6 traffic injected into the stream. Conversely, more expensive hardware-switching systems like the Cisco 7606 could maintain full line-rate throughput even with all-packet-sized packets, and any increase in latency was always characterized as either minor or insignificant irrespective of the IPv4/IPv6 traffic ratio. It was also mentioned in the study that the CPU utilisation during IPv4-only and dual-stack operation was virtually the same, indicating that the application of IPv6 does not naturally impose any higher processing overhead in correctly designed routers.

Section 2 – Results

Automated Multiple Platform Testing Results

At a Glance: Summary of Test Results

- The 7606 platform lost zero frames at 100 percent of 10Gbps line rate (bidirectional) regardless of frame size tested or amount of IPv6 traffic in the test stream.
- On the other test platforms, smaller frame sizes showed a degradation of throughput directly proportional to the amount of IPv6 traffic. The severity of the degradation differed between each platform, with the 7200 being the least severe and the 2811 being the most severe. This is expected because of the relative sizing of the centralized CPU on the respective routers.
- Increasing the amount of IPv6 in the test stream had little impact on the average latency.
- The CPU usage was essentially the same for a dual stack environment as compared to an IPv4 only environment, indicating that dual stack does not impact CPU usage.

Chart Legend

Configuration	Translation
100-0	100% IPv4 traffic 0% IPv6 traffic
90-10	90% IPv4 traffic 10% IPv6 traffic
50-50	50% IPv4 traffic 50% IPv6 traffic
10-90	10% IPv4 traffic 90% IPv6 traffic
0-100	0% IPv4 traffic 100% IPv6 traffic

8

Figure 2.3: *Test result summary of Cisco white paper*

Figure 2.3 demonstrated the result summary for this paper. These results are part of the general picture that IPv6 performance problems are not necessarily protocol-based, but rather attributable to certain implementation aspects. The Cisco results, together with the academic research that reports both positive and negative performance results in real networks, support this claim by demonstrating that protocol-level behaviour must be empirically tested in controlled conditions to isolate protocol-level behaviour and infrastructure-specific constraints. This is in line with the driving force behind the current study that aims at determining the performance of IPv4 and IPv6 during LAN and WAN configurations in the workload of real applications and with the latest virtualised versions.

CHAPTER 3

METHODOLOGY

3.1. Research Design

The research design is based on an experimental quantitative research design to quantitatively compare the performance of IPv4 and IPv6 on a controlled virtual testbed. The methodology is aimed at analyzing two most important metrics of network performance: throughput and round-trip time (RTT). Each of the experiments was conducted in a dual-stack environment, in which both IPv4 and IPv6 are run at the same time and to do a direct comparison under the same network conditions.

The experiment is separated into one control group of testing environments:

- LAN Environment - two immediate virtual PCs (VM1 and VM2) communicate with each other.
- WAN Environment - communication between VM1 and a remote server, which is being routed through two Cisco IOS routers.

Both IPv4 and IPv6 have all the test scenarios run three times to promote statistical reliability. iperf3, FTP, and SMB traffic results are obtained and verified with Wireshark packet captures.

3.2. Network Topology

The network topology is constructed using GNS3, consisting of:

- Two Cisco IOS routers
- Two Ubuntu-based VMs
- One Ubuntu Server

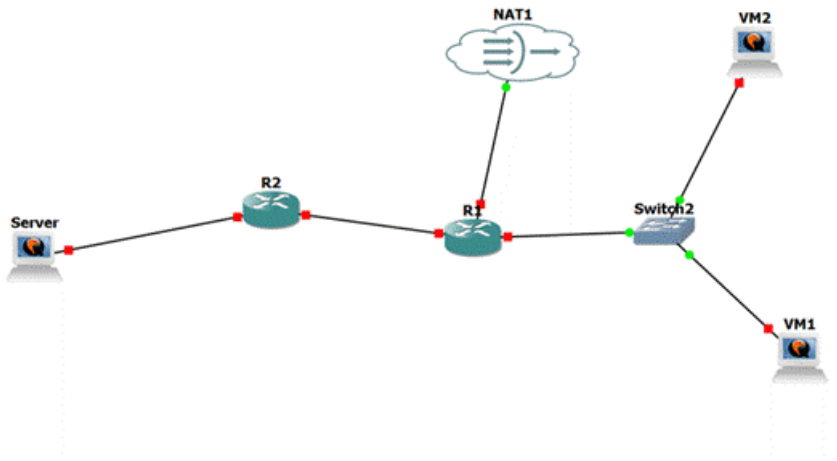


Figure 3.1: *GNS3 network topology*

Figure 3.1 shows the network topology, including 1 Local area network (LAN) connection between VM1 and VM2 inside a same subnet with Router 1 (R1) as default gateway, 2 Wide area network (WAN) connections between VM1, VM2 and server with different subnets, separating by 2 routers.

This topology also includes a Network address translation (NAT) for IPv4 connection to reach the public network

3.3. Protocol detail

3.3.1. File Transfer Protocol

File transfer protocol (FTP) is an old application layer protocol that is tailored to transfer bulk data between hosts through a network. Its main aim is to make the most out of the throughput efficiency in file transmission and hence is highly applicable in the movement of large datasets with minimal protocol overheads. FTP uses a control channel and a separate data channel to send and receive commands and files respectively, and thus partially streams data after a connection is established. Since it is simply constructed, FTP can better make use of bandwidth available, particularly when the network is stable like LANs or controlled WANs.

Practically, FTP is frequently applied in the situation when it is necessary to take a big number of files instead of interactively accessing files. Common applications are systems backup, software distribution, dataset synchronization and content delivery among servers. Because FTP is oriented towards sequential, but not file locking or simultaneous access, it is

especially used in performance benchmarking applications in which the experimenter is interested in measuring the raw network throughput and protocol behavior at scale.

3.3.2. Server Message Block

Server Message Block (SMB) is used on a network-based file sharing and not transferring large amounts of files. It facilitates remote systems to access, read, write and modify files as though they were on the local storage. SMB promotes locking files, exchange of metadata and session management that are critical to a collaborative environment. Such extra control mechanisms are more protocol overhead than FTP, however, they are more consistent and easier to use as a shared file access mechanism.

The SMB is commonly used in enterprise systems that demand centralized storage and shared folders. Common uses are collaborative document editing, shared drives and network-attached storage (NAS) systems. Although the transfer rates are not always as high with SMB compared to FTP, as it is also an interactive system and involves control exchanges, it can also give a realistic picture of daily file access patterns. Due to this reason, SMB can be used in the performance evaluation research that seeks to capture realistic user workloads to supplement the throughput focus that FTP provides.

3.4. Hardware and Software Setup

3.4.1. Virtualization Environment

GNS3 VM

The entire network is deployed inside the GNS3 network simulation platform, which integrates virtual routers, virtual machines, and traffic-capturing tools in a unified workspace.

The GNS3 VM—running on VMware Workstation—is responsible for executing device emulations, providing a stable and isolated environment for the test scenarios.

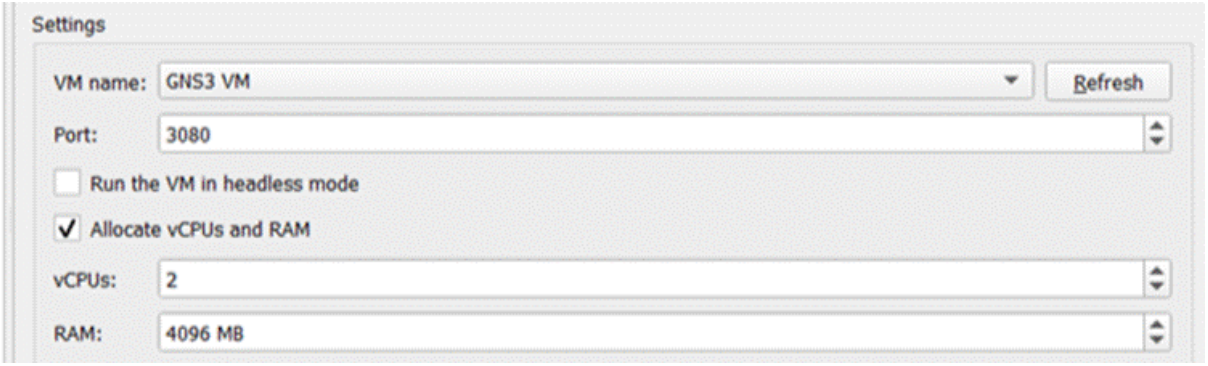


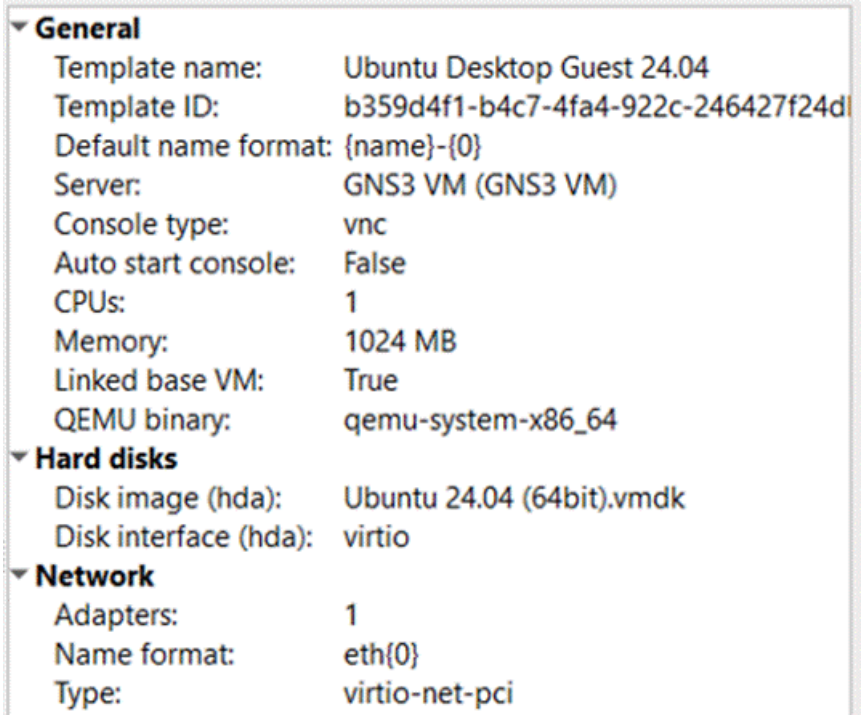
Figure 3.2: *GNS3 VM configuration*

The GNS3 VM is configured as shown in the Figure 3.2:

- vCPUs: 2 cores
- Memory: 4 GB RAM
- Operating Mode: Hardware virtualization accelerated (KVM/QEMU)
- Port: 3080

QEMU virtual machine

Three Ubuntu-based QEMU virtual machines act as the network endpoints for throughput and RTT testing.



General	
Template name:	Ubuntu Desktop Guest 24.04
Template ID:	b359d4f1-b4c7-4fa4-922c-246427f24d
Default name format:	{name}-{0}
Server:	GNS3 VM (GNS3 VM)
Console type:	vnc
Auto start console:	False
CPUs:	1
Memory:	1024 MB
Linked base VM:	True
QEMU binary:	qemu-system-x86_64
Hard disks	
Disk image (hda):	Ubuntu 24.04 (64bit).vmdk
Disk interface (hda):	virtio
Network	
Adapters:	1
Name format:	eth{0}
Type:	virtio-net-pci

Figure 3.3: *QEMU VM configuration*

Hardware specifications shown in Figure 3.3, including:

- CPU: 1 vCPU
- Memory: 1 GB RAM
- Virtual NIC: VirtIO network interface
- OS: Ubuntu Linux 24.04 LTS (dual-stack enabled)

Cisco IOS router

In this study, both routers are implemented as virtual Cisco devices running within the GNS3 environment. Even though they are virtual, they operate using real Cisco IOS images and therefore accurately replicate the behavior of physical enterprise routers.

The routers are configured using Cisco IOS Software, specifically a 7200-series image capable of supporting IPv4, IPv6, static routing, and general Layer 3 forwarding functions.

Each router is assigned the following virtual hardware resources:

- RAM: 512 MiB
- NVRAM: 512 KiB

3.4.2. Device Configurations

Router 1 serves as the initial destination of both the virtual PCs, in between the LAN and the simulated WAN, to make sure that the traffic between VM1 and VM2 will be routed to Router 2 and eventually to the Server. It has three active interfaces: FastEthernet0/0 is connected with NAT1 with IPv4 address 192.168.127.1, which enables Router 1 to convert the local addresses into public IP addresses; GigabitEthernet1/0 is connected to VM1 and VM2 and configured with 192.168.1.1/24 and 2001:db8:1::1/64 which is the default gateway of the two hosts. Router 1 was configured with the static routes that would direct the entire traffic destined to the server networks using interface GigabitEthernet3/0 configured with 192.168.60.1/24 and 2001:db8:60::1/64 to the WAN interface of Router 2, so that either the IPv4 or the IPv6 packets would always leave at the appropriate route.

Router 2 plays the role of linking Router 1 and the Server and it is located on the WAN side of the topology and processes all the forwarding after traffic exits the LAN. It works with two interfaces: GigabitEthernet1/0, set up with 192.168.60.2/24 and 2001:db8:60::2/64, which receives all the traffic sent by Router 1, and GigabitEthernet2/0, which is configured with 192.168.30.1/24 and 2001:db8:30::2/64, which delivers all the packets to the Server. As the Router 2 is in between the LAN and the Server, it should also forward back traffic to VM1 and VM2. To facilitate this, the addition of the static routes was made in the direction of the addresses of the WAN side of Router 1 so that the packets which are sent back can be guaranteed to take the proper route both in IPv4 and IPv6.

The 3 local machines including VM1, VM2 and Server all use the same technique to perform network configuration, which is using the netplan yaml configuration file

```
network:
    version: 2
    ethernets:
      ens3:
        dhcp4: no
        dhcp6: no
        addresses:
          - 192.168.1.2/24
          - 2001:db8:1::a/64
        gateway4: 192.168.1.1
        gateway6: 2001:db8:1::1
        nameservers:
           addresses: [8.8.8.8, 1.1.1.1]
```

Figure 3.4: *VM1 network configuration*

Figure 3.4 illustrates the content of the netplan configuration file for VM1. As shown, the host uses network version 2 ethernet with no DHCP to manually configure the private IP address, the static IP address and the default gateway IP address for both IPv4 and IPv6.

3.4.3. Device Configurations

Table 2: Performance measurement tools

Tool	**Purpose**
iPerf3	Controlled TCP throughput measurement
vsftpd (FTP)	Application-layer throughput test
Samba (SMB)	File-sharing throughput test

Wireshark	Packet capture, throughput and RTT extraction, TCP issue detection

Table 2 shows the necessary tools and software when this test was conducted.

iPerf3 is a network performance measurement utility to create controlled TCP or UDP traffic to measure the throughput, retransmissions and congestion behaviour between two endpoints. It is very popular in the benchmarking of the performance of raw transport-layers.

vsftpd (FTP) is a small but secure FTP server which is applied to transfer files via TCP. It offers throughput measurements of the application-layer in performance testing that are real-world file download behavior.

Samba also offers the application of Server Message Block (SMB)/Common Internet File System (CIFS) protocol in sharing files among network-enabled devices. When testing it, it measures throughput when subjected to a standard enterprise file-sharing workload with more protocol overhead than FTP.

Wireshark is a packet capture and analysis application used to trace network live traffic and inspect TCP streams, throughput profiles, and round trip time metrics as well as network problems like retransmissions, out of order packets and so on.

3.5. Test Procedures

3.5.1. General Test Steps

In testing the protocols, iPerf3, FTP, and SMB, the process used was the same in order to obtain similar and comparable results. The Wireshark was used to initiate packet captures on the respective VM interface and then each test was executed three times on IPv4 and again on IPv6. Major performance data was logged during each run, with instantaneous and average throughput, per-packet RTT, and all TCP-related events, including retransmissions, fast retransmissions, duplicate ACKs, out-of-order packets, and window-full events.

3.5.2. iPerf3 Test Procedure

Client commands:

Both the IPv4 and IPv6 client use the command iperf3 -c <server-ip> to start an iPerf3 client requests with default parameters. The default test will last for 10 seconds and output to the terminal of both client and server machines.

```
osboxes@osboxes:~$ iperf3 -c 192.168.30.200
Connecting to host 192.168.30.200, port 5201
[  5] local 192.168.1.2 port 47162 connected to 192.168.30.200 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.01   sec  2.25 MBytes  18.8 Mbits/sec   11    189 KBytes
[  5]   1.01-2.00   sec  1.75 MBytes  14.7 Mbits/sec    0    223 KBytes
[  5]   2.00-3.00   sec  1.12 MBytes  9.43 Mbits/sec    0    245 KBytes
[  5]   3.00-4.00   sec  1.62 MBytes  13.6 Mbits/sec    0    256 KBytes
[  5]   4.00-5.03   sec  1.25 MBytes  10.2 Mbits/sec    4    191 KBytes
[  5]   5.03-6.00   sec  1.12 MBytes  9.71 Mbits/sec    4    147 KBytes
[  5]   6.00-7.00   sec  1.12 MBytes  9.44 Mbits/sec    0    163 KBytes
[  5]   7.00-8.00   sec  1.62 MBytes  13.6 Mbits/sec    0    171 KBytes
[  5]   8.00-9.00   sec   640 KBytes  5.25 Mbits/sec   20   87.7 KBytes
[  5]   9.00-10.00  sec  1.62 MBytes  13.6 Mbits/sec    0   99.0 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  14.1 MBytes  11.8 Mbits/sec   39             sender
[  5]   0.00-10.07  sec  13.0 MBytes  10.8 Mbits/sec                  receiver
```

Figure 3.5: *iPerf3 client testing*

Figure 3.5 demonstrates a process of iPerf3 testing the TCP connection. Rather than transmitting actual files, iPerf3 creates and exchanges structured pseudo-random data blocks. This ensures consistent, repeatable testing that is independent of application-layer overhead.

3.5.3. FTP Test Procedure

Client commands:

```
osboxes@osboxes:~$ ftp 192.168.30.200
Connected to 192.168.30.200.
220 (vsFTPd 3.0.5)
Name (192.168.30.200:osboxes): osboxes
331 Please specify the password.
Password:
```

Figure 3.6: *vsftpd client authentication*

Connecting to the vsftpd server requires using the command ftp <server-ip>. As Figure 3.6 shows, after entering the command, there will be a prompt for the user to enter the name and password of the server. If the authentication is passed, the user will be navigated to vsftpd shell

```
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> get python-3.12.0-amd64.exe
local: python-3.12.0-amd64.exe remote: python-3.12.0-amd64.exe
229 Entering Extended Passive Mode (|||54993|)
150 Opening BINARY mode data connection for python-3.12.0-amd64.exe (26507904 by
tes).
100% |***********************************| 25886 KiB    1.38 MiB/s    00:00 ETA
226 Transfer complete.
26507904 bytes received in 00:18 (1.37 MiB/s)
```

Figure 3.7: *vsftpd client retrieve data*

The Figure 3.7 illustrates the vsftpd shell after the user successfully logged in the server. The user can now use several commands such as ls to list all files, get file to retrieve a file or put file to upload a file to server.

Large file transfers are performed to generate stable throughput.

3.5.4. SMB (Samba) Test Procedure

The Server Message Block protocol (SMB) was tested using Samba

The samba server has a shared directory in its file system so every client that has access to that server can view the shared content. The client uses the command smbclient //<server-ip>/<directory-name> -U username to access the samba server. After entering the command, the user will be prompted to authentication using server password.

```
Try "help" to get a list of possible commands.
smb: \> ls
  .                                   D        0  Sun Nov 30 06:11:53 2025
  ..                                  D        0  Sun Nov 30 06:11:53 2025
  testfile.txt                        N       38  Sun Nov 30 06:11:10 2025
  python-3.12.0-amd64.exe             N 26507904  Sun Nov 23 15:59:32 2025

                514934016 blocks of size 1024. 472039608 blocks available
smb: \> get python-3.12.0-amd64.exe
getting file \python-3.12.0-amd64.exe of size 26507904 as python-3.12.0-amd64.ex
e (1158.3 KiloBytes/sec) (average 1158.3 KiloBytes/sec)
```

Figure 3.8: *smb client retrieve data*

Similar to vsftpd, the user after authentication will have a smb shell that can perform actions such as lists, retrieve or upload files from the shared directory. The Figure 3.8 demonstrates the actions of listing all existing files inside the shared directory and an attempt to retrieve one of those files

3.5.5. Wireshark Analysis Procedure

The wireshark capturing was done for each protocol of each connection (LAN and WAN), with each protocol tested 3 times on both IPv4 and IPv6. The analysis procedure was conducted using 3 types of graph: I/O, Throughput and Round-trip time (RTT) graphs.

Figure 3.9: *Wireshark I/O graph*

As shown in the Figure 3.9, the I/O graph general setting was set with interval 100ms, Y-axis as bits to measure throughput at bits/100ms, filtered packets limited to packets of the main TCP stream.

The TCP issues include some of the most common TCP errors found during the capture: retransmission, fast retransmission, duplicate ACK, out-of-order packet and window-full.

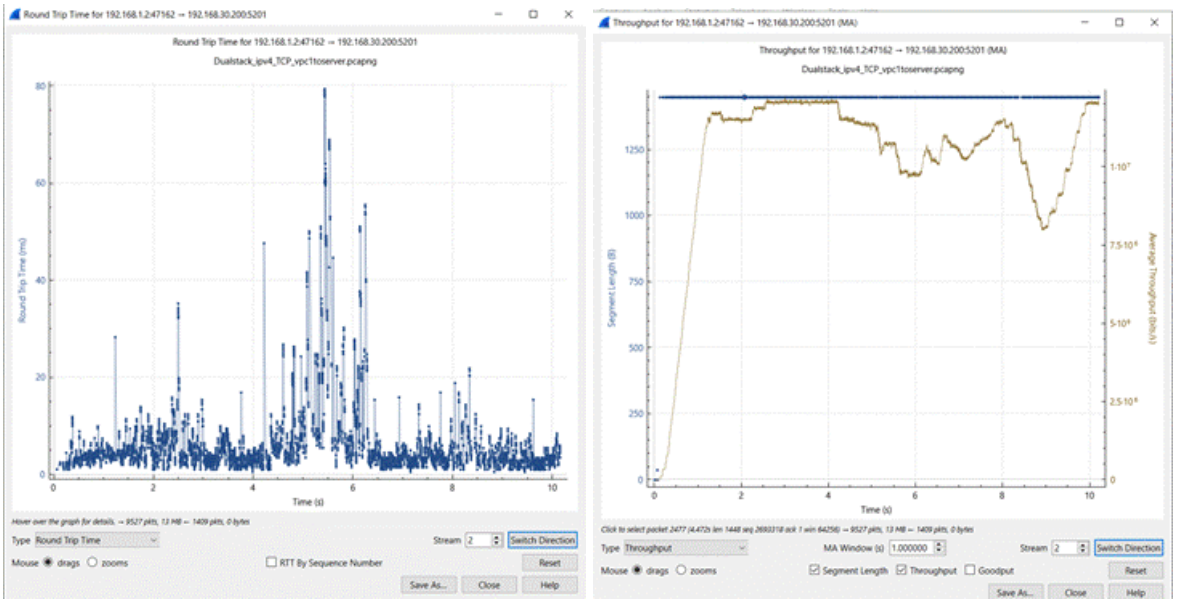


Figure 3.10 & 3.11: *Wireshark RTT and throughput graph*

Both round-trip time graph (3.10) and throughput graph (3.11) use the default setting for clear and fair comparison. The graph will only illustrate the traffic of the main TCP stream.

3.6. Throughput and Round-Trip time calculation

3.6.1. Average throughput calculation

Throughput represents the rate at which data is successfully delivered over the network and is fundamentally defined as:

$$Throughput = \frac{Total\ data\ transferred\ (bits)}{Total\ transmission\ time\ (seconds)}$$

In practice, in network measurements, traffic is not sent at a rate that is perfectly constant. Instantaneous variation may be brought about by short bursts, TCP congestion control behavior, retransmissions, and queueing delays. This necessitates the use of fixed-duration time bins to compute throughput in this study as they are a more realistic time-dependent signal that offers a more smooth throughput.

A packet trace is initially ordered by time, and the total time taken in the capture is broken into fixed length slices also known as time bins (ex: 0.5 or 1 s). Throughput of time bin iii is given by:

$$Throughput_i = \frac{\Sigma Packet\ size_i\ x\ 8}{\Delta t}$$

where:

- $\sum$Packet Size$_i$ is the total number of bytes observed within time bin i,
- 8 converts bytes to bits,
- Δt is the bin width in seconds.

This method is precisely conventional in as far as the original definition of throughput, and it is more robust in the sense that the effect of temporary spikes or downtime is reduced. Time binning is specifically significant in WAN contexts where the flexible nature of latency and TCP window modifications can distort real time throughput measurements otherwise.

In the implementation employed in this work shown in the code in Figure 3.12, the length of packets is summed up in a time bin, multiplied by bits per second and lastly adjusted back to Mbps. The approach guarantees that the values of throughputs indicate the performance which is sustained and not the transmission artifact which is immediate

```
bin_width = 1.0
steady_fraction = 0.6

df = pd.read_csv(csv_path)

df = df[[time_col, len_col]].dropna().copy()
df = df.sort_values(by=time_col).reset_index(drop=True)

start = df[time_col].iloc[0]
end   = df[time_col].iloc[-1]
bins = np.arange(start, end + bin_width, bin_width)

df['bin'] = pd.cut(df[time_col], bins=bins, right=False, labels=bins[:-1])

with warnings.catch_warnings():
    warnings.simplefilter("ignore", FutureWarning)
    agg = df.groupby('bin')[len_col].sum().reset_index()
agg['bin'] = agg['bin'].astype(float)
agg = agg.rename(columns={len_col: 'bytes_in_bin'})

agg['bits_per_s'] = agg['bytes_in_bin'] * 8.0 / bin_width
agg['Mbps'] = agg['bits_per_s'] / 1e6

avg_bps = agg['bits_per_s'].mean()

print(f"Avg: {avg_bps:.2f} bps ({avg_bps/1e6:.4f} Mbps)")
```

Figure 3.12: *Python code to calculate average throughput*

3.6.2. Average round-trip time (RTT) calculation

The time calculated round-trip was based on actual RTT samples obtained out of the measurement datasets. The RTT values are the time durations that a packet took to be sent and the time duration to receive the corresponding acknowledgement. Mean RTT per run was calculated by computing an arithmetic mean of all the RTT samples that were recorded in the dataset.

```
df = pd.read_csv("iPerf3_IPv6_VPC1toServer_R3R4R5.csv")

rtt = df["RTT"]

print("Avg RTT:", rtt.mean())
```

Figure 3.13: *Python Code to extract the average Round-trip time*

This method gives a brief depiction of the end-to-end latency features of every routing path. With averaged RTT values, it is possible to reduce transient spikes due to congestion or retransmissions and consequently better compare routing behaviour and protocol performance across various experimental conditions.

CHAPTER 4

RESULT AND ANALYSIS

4.1. LAN topology

The topology includes 2 hosts connected in a same subnet, the IPv4 and IPv6 of both machines follow:

- VM1: 192.168.1.10/24, 2001:DB8:1::A/64
- VM2: 192.168.1.20/24, 2001:DB8:1::B/64

4.1.1. iPerf3

I/O graph:

Figure 4.1: *I/O graph of iPerf3 LAN capture during the 3rd run*

The I/O curves of the iPerf3 LAN tests indicate that TCP stability is nearly similar between IPv4 and IPv6 with both protocols experiencing predictable congestion-window cycles and the only significant difference being in the count of recovery events. The IPv4 had 14 TCP issues in the first run and IPv6 had 15 and nearly the same behaviour. This was also the same case in the second run with 17 issues with IPv4 and 18 with IPv6. The third re-run as shown in Figure 4.1 indicated the same with IPv4 reporting 19 TCP problems and IPv6 reporting 18.

Compared to the IPv4, when calculated across all the runs, the IPv4 gives an average of 16.7 problems and the IPv6 only 17.0 which are basically the same in a LAN setup.

In total runs, IPv4 has the average of 16.7 issues, and IPv6 has the average of 17.0 issues, which is a negligible difference of 0.3 events. This insignificant variation proves that IPv6 imposes no quantifiable cost to TCP stability or disruption of losses in the managed LAN environment. This is in line with Soorty and Sarkar (2013) findings in which IPv6 congestion-control behaviour trailed IPv4 closely in the low-loss Gigabit Ethernet.

Throughput graph:

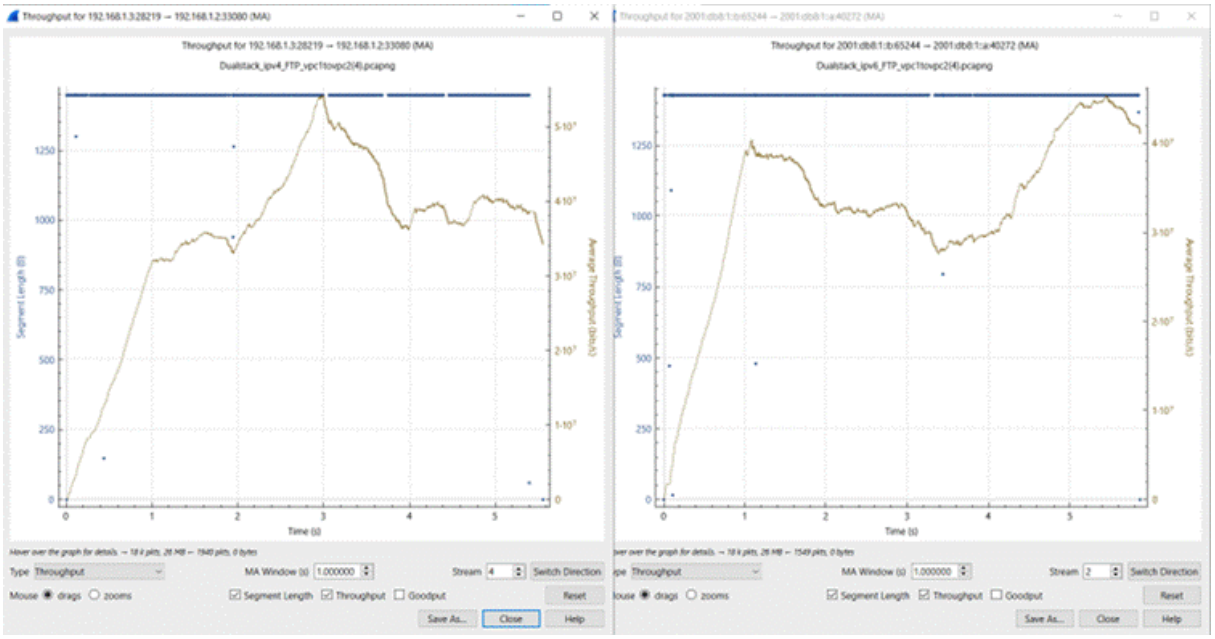


Figure 4.2: *Throughput graph of iPerf3 LAN capture during the 3rd run*

Throughout the iPerf3 LAN test throughput tests, IPv4 had a higher transfer rate than IPv6, although by different amounts depending on the test run. During the initial run, the average throughput of IPv4 was 38.65 Mbps as compared to 37.05 Mbps. The second run was also similar with the average IPv4 of 39.89 Mbps and IPv6 of 38.05 Mbps. The greatest difference was in the third run illustrated in Figure 4.2 in which IPv4 had 41.89 Mbps over IPv6 which had 41.06 Mbps. When all these runs are added together, IPv4 has a minuscule yet consistent benefit, with an average of 40.14 Mbps over 38.72 Mbps of IPv6.

The variation in performance between all the runs is the same thus implying that the throughput penalty with IPv6 under the same test condition is constant. Despite the LAN environment not adding routing overhead and insignificant latency, the increased IPv6 header size causes a slight drop in payload efficiency, also explaining the systematic but insignificant decrease in throughput.

It is interesting to note that IPv6 can be seen to reach greater instantaneous peaks, such as 48.38 Mbps in Run 3, which means that IPv6 is not necessarily constrained in short-term bursts. The sustained throughput is however slightly lower, showing that it is not a temporary occurrence on specific intervals of the test.

In general, measurements of throughput in all of the runs indicate that IPv6 operates almost identically to IPv4, only a slightly predictable decrease in the effective data rate.

Round-trip time graph:

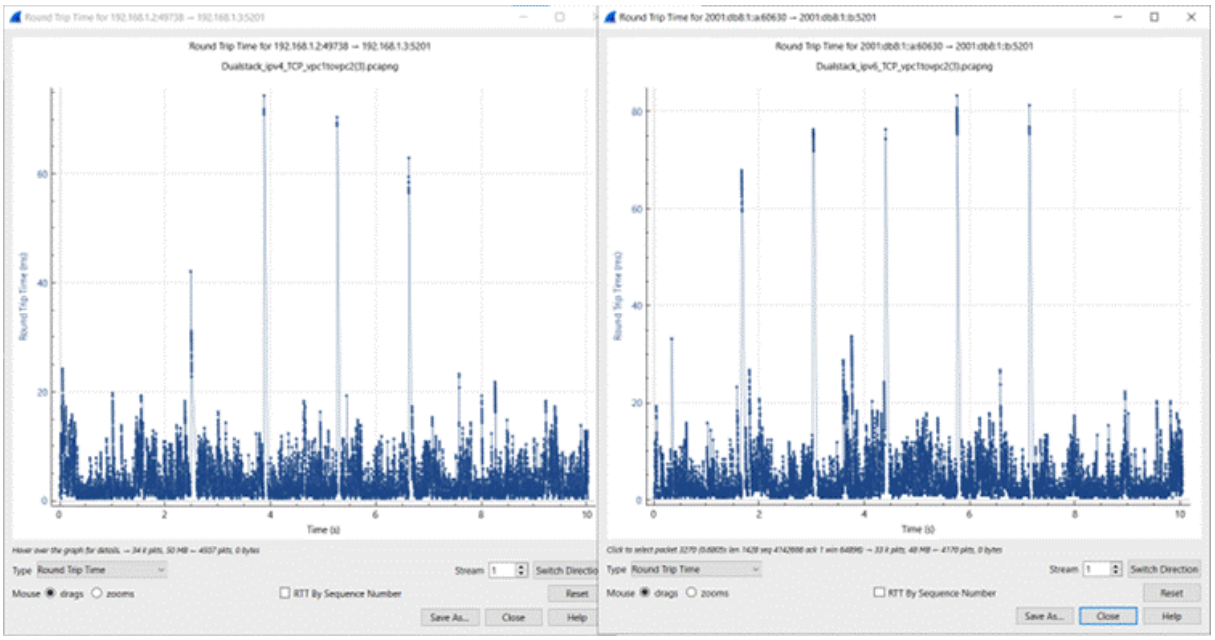


Figure 4.3: *RTT graph of iPerf3 LAN capture during the 3rd run*

The tests for iPerf3 LAN showed very narrow concentration in the RTT behaviour of both protocols, and IPv4 typically had slight lead over average delay. In the initial run, IPv4 indicated an average RTT of 0.00168 seconds and IPv6 indicated a larger value of 0.00219 seconds which indicates some other delay spikes. This trend was repeated in the second run as IPv4 was running at 0.00251s and IPv6 ran at 0.00207s with IPv6 briefly surpassing IPv4.

Figure 4.3 shows that the third run resorted once again to IPv4 that took an average time of 0.00213 seconds against IPv6 that took 0.00233 seconds. In all the runs, IPv4 had a general average RTT of approximately 0.00211 seconds whereas IPv6 had a general average of approximately 0.00220 seconds.

The higher maxima of IPv6 indicate that in occasional cases, there is either the presence of queueing or delaying of the ACK interval, which is marginally more common with IPv6, but the maxima do not have a significant impact on the median values, which are closely clustered with both protocols. The total amount of latency difference to be referred to is slim but detectable, and it is in line with the fact that the IPv6 packets are slightly larger, and the processing per segment is minimal.

4.1.2. FTP (VSFTPD)

I/O graph:

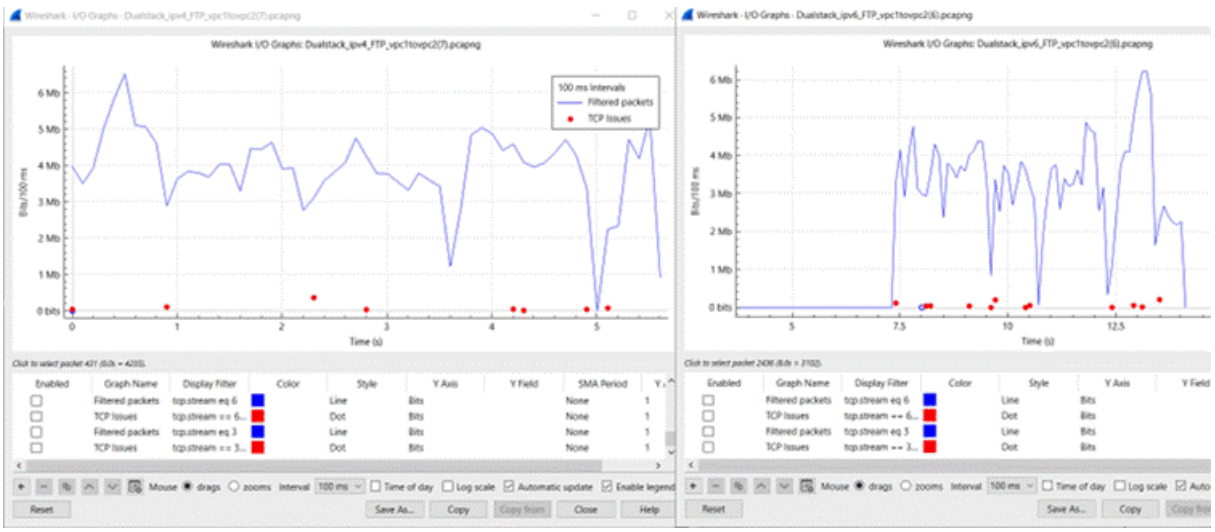


Figure 4.4: *I/O graph of FTP LAN capture during the 2nd run*

TCP behaviour of FTP on the LAN is the same with the IPv4 and IPv6, the only difference is run-to-run differences in events due to congestion. In the initial test run, IPv4 had 9 TCP problems and IPv6 had 7 which means that IPv6 had a slightly smoother transmission in that case. The trend was inverted in the second run as IPv4, as shown in Figure 4.4, was faced with 8 problems, and IPv6 was faced with 12 problems, indicating an interim growth in retransmission exercises among IPv6. In the third run, the two protocols were once again closer with IPv4 recording 9 issues and IPv6 recording 9 issues respectively. Mean response

times averaging all the three test runs, there are approximately 8.67 TCP problems with IPv4 and 9.33 problems with IPv6, a difference essentially negligible.

The difference of 0.6 events is negligible and does not suggest that there is any significant divergence in TCP stability. In spite of the fact that IPv6 is a bit more variable, particularly in Run 2 in Figure 5.4, where problems peak at 12, this variability is not persistent across runs and is not associated with severe performance impairment.

Generally, congestion-related events are similar in both IPv4 and IPv6 traffic using FTP implying that the character of loss of both protocols is almost similar and that both protocols are not structurally different in behaviour with regard to congestion control or recovery in a LAN.

Throughput graph:

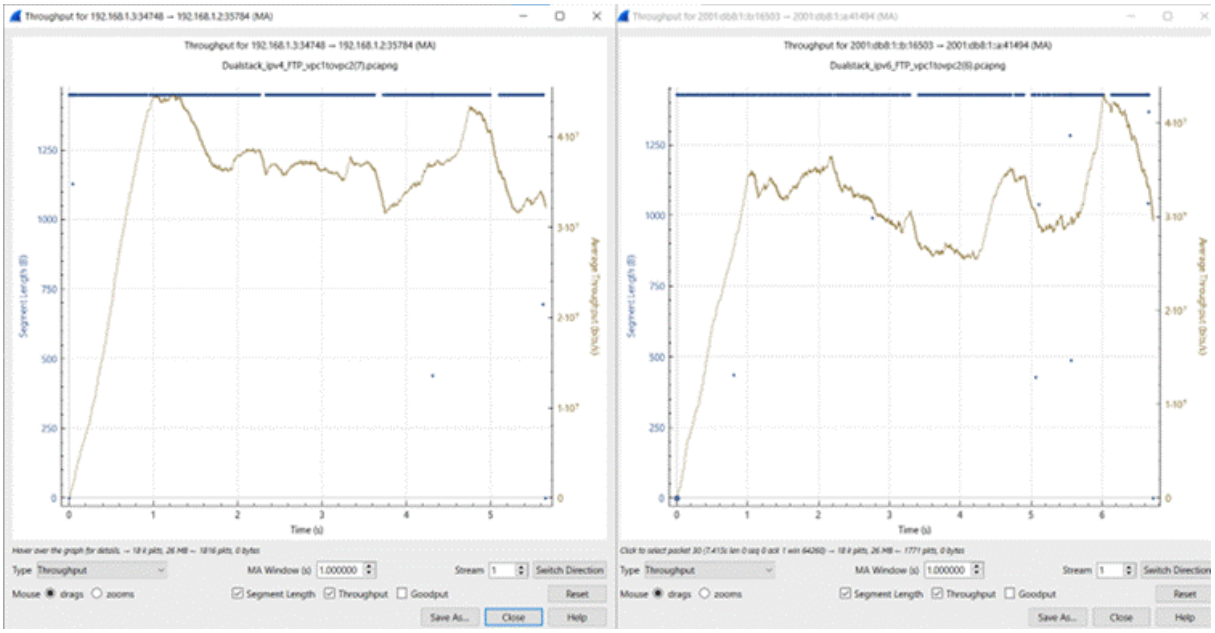


Figure 4.5: *Throughput graph of FTP LAN capture during the 2nd run*

In all the FTP LAN tests, IPv4 reported higher throughput than IPv6 with performance difference clearly and repeatedly evident. As a first-time performance, IPv4 recorded an average of 43.24 Mbps, which is significantly higher than 37.65 Mbps of IPv6. The second run, as displayed in Figure 4.5, recorded the same trend whereby IPv4 had an average of 40.59 Mbps and IPv6 had 35.18 Mbps. During the third run, both protocols worked closer to each other but IPv4 had slightly better results with 44.30 Mbps than IPv6 with 43.36 Mbps.

With a combination of all the runs, the IPv4 gives an overall average throughput of 42.71Mbps with IPv6 having 38.73 Mbps; IPv4 has an enormous edge when it comes to FTP transfers over the LAN.

The throughput gap between the tests on the FTP is greater in comparison with the iPerf3 measurements. This is probably because of protocol-layer overheads in FTP which use more interaction in the control channel, and also have a more frequent conversion of states than a simple bulk transfer stream using TCP. Since FTP has more frequent metadata interactions, the larger header of IPv6 has a proportionately greater impact on overall payload efficiency.

It is also interesting to note that IPv6 also has the least throughput in run 2 (35.18 Mbps) which is the run that has the highest number of IPv6 TCP issues (12). Although this is not a systematic trend in all of the experimental run times, it does hint at the possibility that IPv6 is somewhat more prone to temporary throughput dips during heavier state-exchange workloads.

However, IPv6 also remains competitive in the entire dataset, with even near-parity in Run 3 with IPv4 (44.30 Mbps vs. 43.36 Mbps). This shows that the throughput disadvantage is minimal in terms of absolute sense and is within predictable and constant limits.

Round-trip time graph:

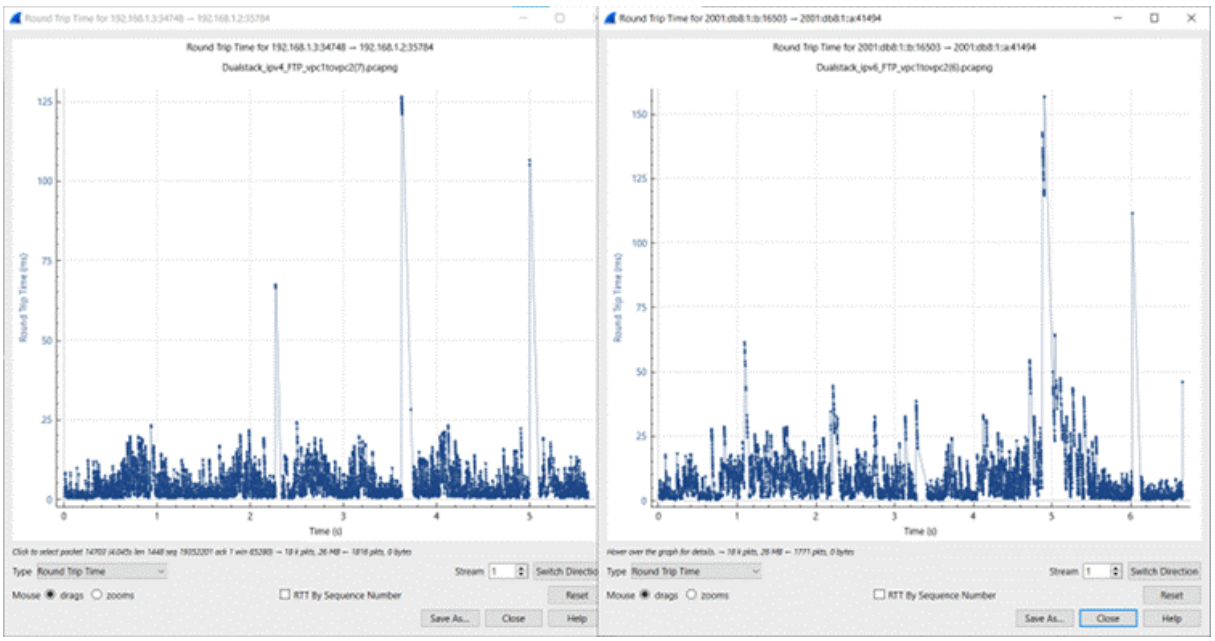


Figure 4.6: *RTT graph of FTP LAN capture during the 2nd run*

FTP measurements to the LAN portray an erratic trend wherein both protocols alternate in every run yet generate low and constant delay in general. The average RTT of IPv6 was marginally lower than that of IPv4 in the initial run at 0.00196 vs. 0.00213 s respectively, which indicates a cleaner control-channel exchange. The second run, demonstrated by Figure 4.6, overturned this behaviour where the IPv4 took 0.00194 s on average and the IPv6 increased greatly to 0.00389 s by a tiny group of delay spikes. The third run was back to almost parity with IPv4 having an average RTT of 0.00209 s and IPv6 having an average RTT of 0.00211 s. When all the runs are added together, the IPv4 mean RTT is approximately 0.00205 s whereas the IPv6 mean RTT is about 0.00265 s, with IPv4 having a slight advantage of latency on the LAN.

Both protocols have low baseline RTTs though the IPv6 protocols have higher averages and more medians, reflecting a slight yet significant latency penalty when transferring data between FTP. The RTT max values are also similar between the two protocols and this indicates that the various protocols do not have extreme tail-latency events in the LAN environment

4.1.3. SMB (Samba)

I/O graph:

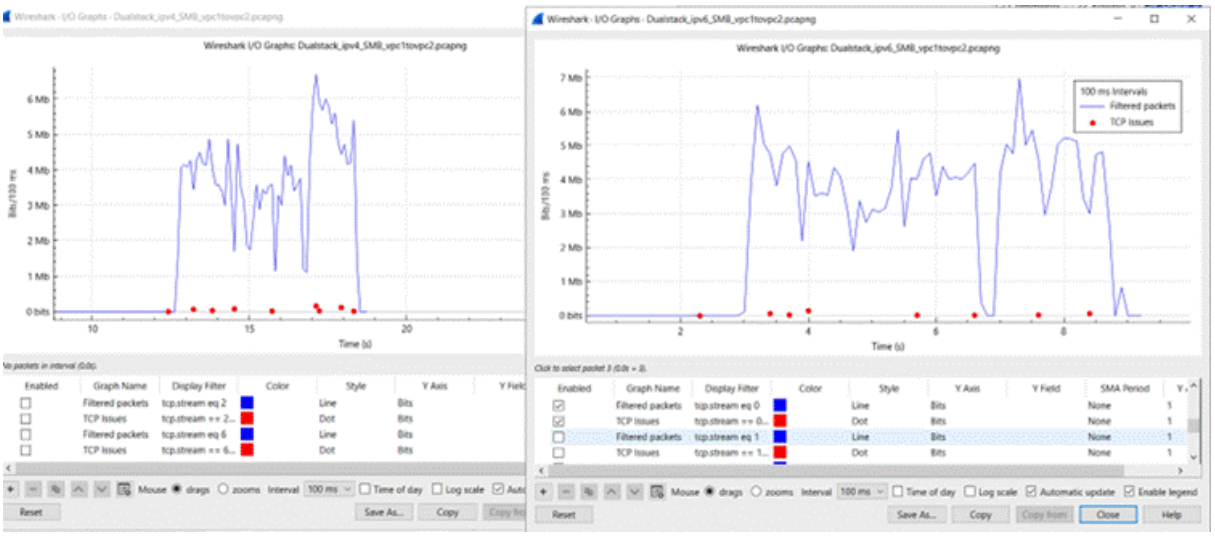


Figure 4.7: *I/O graph of SMB LAN capture during the 1st run*

Throughout the SMB LAN tests, TCP behaviour was consistent and closely matched between IPv4 and IPv6 with only large variations in the amount of congestion-related events per run.

The results of the initial test run as shown in Figure 4.7 displayed IPv4 with 9 TCP problems and IPv6 with 8 problems indicating almost the same rate of retransmission activity. The second run had the opposite meaning in that IPv4 faced 8 problems and IPv6 faced 9 problems, once again showing the proximity with which the protocols interacted in the context of LAN. This pattern of slight variation was again repeated in the third run where IPv4 again registered 8 issues and IPv6 showed a little higher number of 10. The combination of these results has IPv4 averaging 8.3 TCP issues per run versus IPv6 having 9.0 issues per run, a difference of slightly less than a single event and too small to indicate any sort of deviation in congestion-control behaviour.

Although these small run-to-run variations are present, both protocols had smooth congestion-window evolution during the transfers, and saw-tooths were clean and no indications of sustained loss, stuck windows, or instability were evident. The extra recovery signal that was periodically detected under IPv6 did not create any regular pattern and seemed rather to be a result of a short-term fluctuation in timing that characterizes the fast sequence of metadata and acknowledgment messages of SMB. Similarly, the marginally smaller number of events experienced under IPv4 was also isolated and did not have any significant effect on throughput and responsiveness.

All in all, these SMB LAN I/O graphs assure that both IPv4 and IPv6 can equally well manage the interactive workload of the protocol. The low rate of event of the congestion during all runs and the almost similar averages show that the LAN environment offers stable conditions of the use of both the protocols and there are no indications that any of them may be facing higher pressure of retransmission or there are no indications that they are facing high pressure of timing.

Throughput graph:

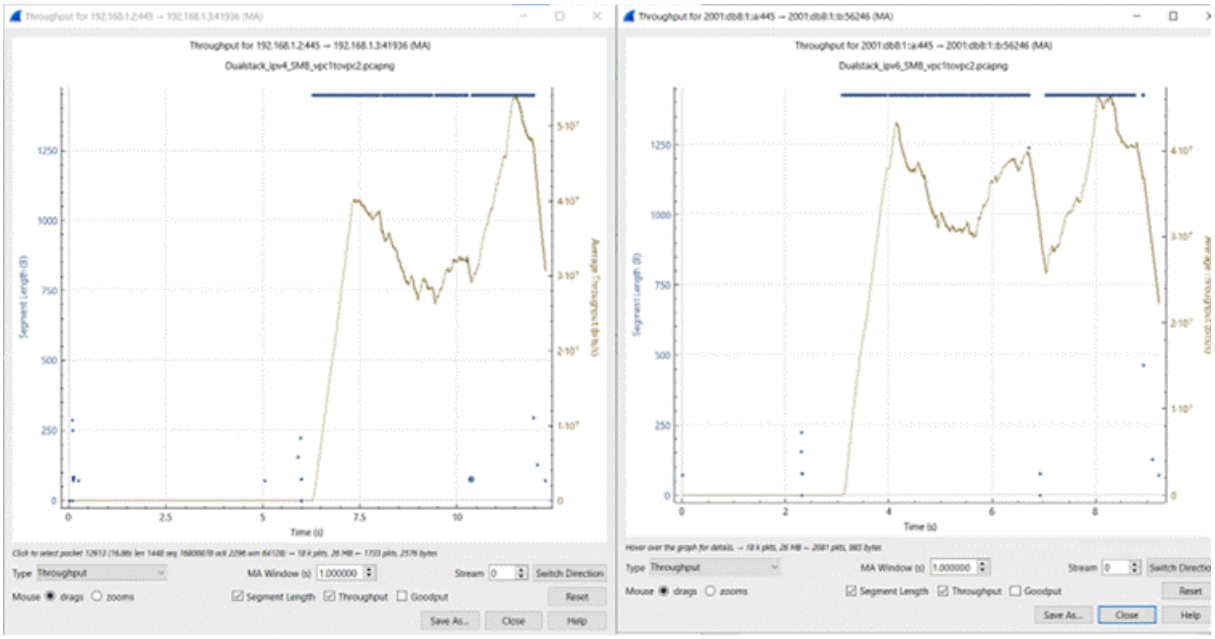


Figure 4.8: *Throughput graph of SMB LAN capture during the 1st run*

On throughput, IPv4 and IPv6 both provide a high throughput with minor differences between them. In Run 1 shown in Figure 4.8, IPv6 was with an average of 37.71 Mbps, a little higher than that of IPv4 of 37.11 Mbps. In Run 2, the difference between IPv6 and IPv4 was more evident with IPv6 of 51.01 Mbps, as opposed to the 46.45 Mbps of IPv4. Run 3 showed IPv4 with a minor lead of 44.55 Mbps and IPv6 with an average of 42.12 Mbps.

The total average throughput of IPv4 is 42.37 Mbps and that of IPv6 is 43.62 Mbps, which is a small difference of 1.25 Mbps (2.9% higher). This trend indicates that there is no regular protocol dominating across the LAN; minor differences in timing, buffering and command sequence will have the lead alternating between run to run. The combination of large-block transfers and fast metadata interactions in SMB implies that small timing variations may affect the overall throughput.

In general, both IPv4 and IPv6 provide high and consistent performance through the LAN, with IPv6 having a minor, though not always reproducible, throughput advantage.

Round-trip time graph:

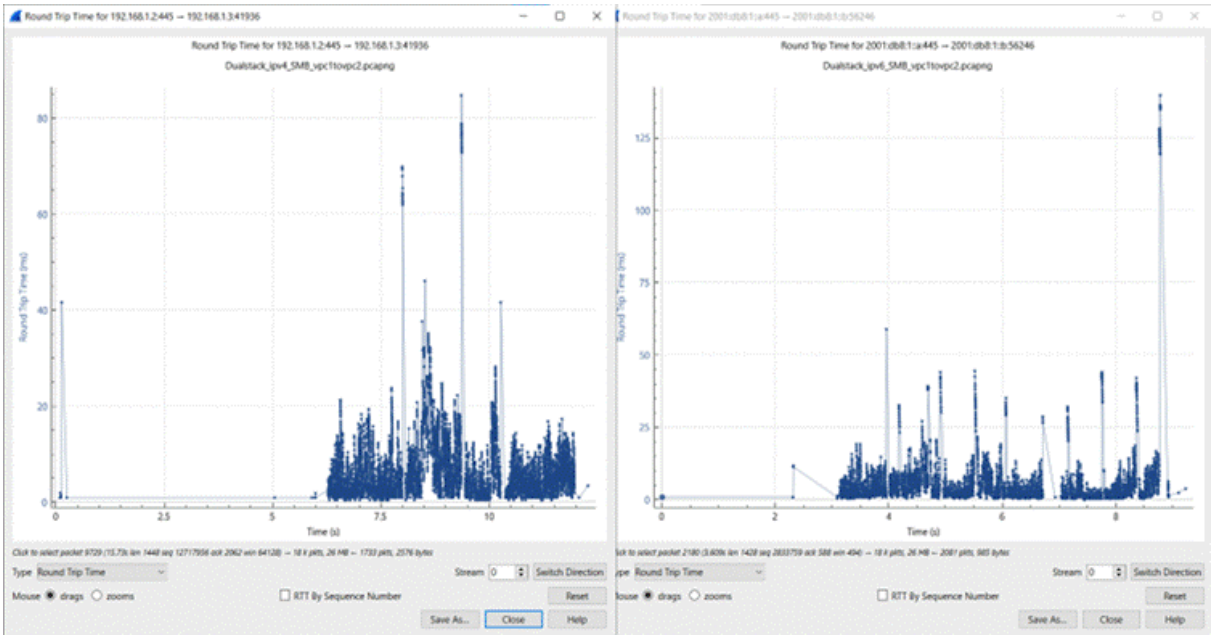


Figure 4.9: *RTT graph of SMB LAN capture during the 1st run*

The SMB LAN RTT measurements show that IPv4 and IPv6 have overall low values of delay in all test runs, with moderate values of run to run variation, but no sign of consistent latency instability. During the initial test run demonstrated in Figure 4.9, the IPv6 as compared to IPv4 had lower average RTT with an average delay of 0.00230 s against IPv4 which had an average delay of 0.00284 s. The second run however depicted the opposite of this, where the IPv4 had a better average RTT of 0.00181 s compared to the IPv6 that was 0.00201 s. During the third run IPv6 showed a very minor latency difference with an average RTT of 0.00196 s, in comparison with IPv4 which had 0.00207 s.

In a combination of all 3 test-runs, IPv6 has a slightly lower mean RTT total in SMB LAN transfers than those of IPv4. Although the individual runs switch between which protocol run is doing better, the differences are small and stay within a rather small latency margin. Such findings suggest that SMB traffic across a LAN has no systematic penalty of latency when using IPv6. Rather, the IPv4 and IPv6 share common RTT behavior with subtle differences due to the transient effects of congestion and application-level interaction patterns and not inherent protocol differences.

4.2. WAN topology

The topology includes 3 hosts connected in different subnets. For this simulation, VM1 was used for communicating to Server only, and the IPv4 and IPv6 of both machines follow:

- VM1: 192.168.1.10/24, 2001:DB8:1::A/64
- Server: 192.168.30.200/24, 2001:DB8:30::200/64

4.2.1. iPerf3

I/O graph:

Figure 4.10: *I/O graph of iPerf3 WAN capture during the 1st run*

When comparing the three runs of the WAN tests, the TCP problems as observed in the iPerf3 I/O graphs show a steady trend with IPv4 showing a few more retransmission/ congestion related events as compared to IPv6. In run 1 as shown in Figure 4.10, IPv4 had 8 TCP problems whereas IPv6 had 2. The result of Run 2 was even, as both the protocols had 5 issues each. Run 3 showed 6 problems with IPv4 as opposed to 5 with IPv6. IPv4 has 6.3 TCP problems per test on average over all three of the test runs, compared to 4.0 problems per test in IPv6, which provides a slight advantage of IPv6 in terms of loss-recovery behaviour.

These findings suggest that the WAN path is more prone to variability and timing instability than the LAN environment that was used in the experiment earlier, causing, on average, increased counts of TCP issues on both protocols. This growth notwithstanding, IPv6

invariably exhibits a wee bit smoother congestion-window profile as indicated by the reduced overall number of retransmissions or recovery events. The fact that IPv4 occasionally equals or nearly equals IPv6 in per-run fluctuations implies that it is unlikely that there is some underlying protocol level performance difference causing these differences, although it may be due to short-term queueing or bursty delay.

In general, the size of the difference is not so big, and both protocols do not exhibit any evidence of constant congestion or unreliable recovery behavior during the WAN tests. IPv4 and IPv6 have stable transfer behaviour and the fact that the difference in the number of TCP issues is neglectable supports the conclusion that the dynamics of the WAN is the dominant factor in the reliability properties of the I/O graphics, rather than the IP version.

Throughput graph:

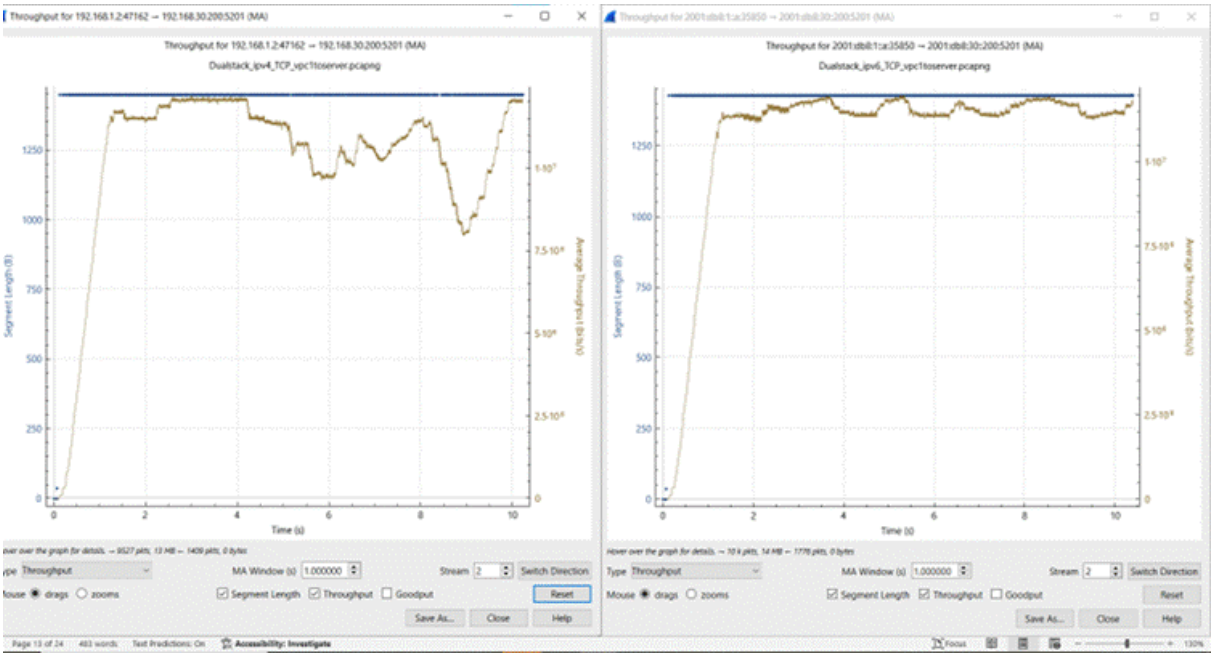


Figure 4.11: *Throughput graph of iPerf3 WAN capture during the 1st run*

Compared to the LAN environment, the iPerf3 WAN tests show a significantly smaller throughput difference between the IPv4 and IPv6 protocols, and the performance switches across the three runs with slightly different results. The initial test of the WAN illustrated in Figure 4.11 showed that IPv6 is faster than IPv4, with that the average throughput was 12.10 Mbps as opposed to 11.36 Mbps of IPv4. The second iteration indicated the close parity of the two protocols. IPv4 had slightly higher average throughput at 11.99 Mbps and the IPv6 had

11.92 Mbps. In the third run, IPv4 gained some slight lead and the average throughput of 11.84 Mbps, where IPv6 had 11.81 Mbps.

In the case of the three runs being compared, the IPv6 has the highest total throughput at about 11.94 Mbps compared to the IPv4 total throughput of 11.73 Mbps. As opposed to the LAN measurements, where IPv4 was dominant all the time, the WAN iPerf3 results indicate that IPv6 is equally competitive and in some of the runs, even better. This is due to the character of iPerf3 that produces a continuous stream of TCP data with little protocol overhead and thus lessening the effect of the increased size of the IPv6 header in longer distance connections.

Round-trip time graph:

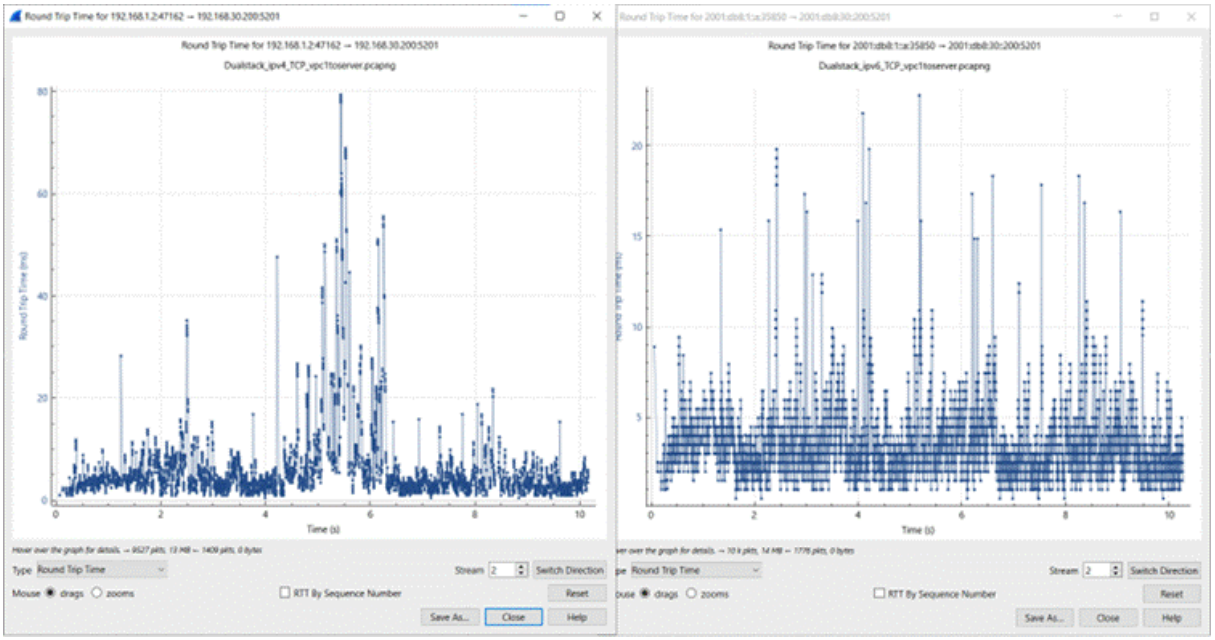


Figure 4.12: *RTT graph of iPerf3 capture during the 1st run*

Measured values of RTT over the WAN indicate that there is always a higher latency than in the LAN setting, as is to be noted, but the values are consistent and under control in both protocols. The initial run shown in Figure 4.12 had a significantly lower average RTT of IPv6 of 0.00312 s than 0.00501 s of IPv4. This trend was slightly reversed in the second run, though the IPv4 had lower average RTT at 0.00391 s, and IPv6 was marginally high to 0.00409 s. During the third run, IPv6 showed a lower latency percentage again with an average RTT of 0.00299 s, as opposed to 0.00427 s in IPv4.

Combining all the runs, IPv6 has the lower overall mean RTT which is about 0.00340 s and in comparison with IPv4, the mean is about 0.00440 s. Both protocols provide decent yet consistent WAN latency, but IPv6 performance is always somewhat better in terms of RTT properties, both in averages and susceptibility to high-latency events. This implies that IPv6 could have a small latency benefit on long-term data transfers in WAN when plain-vanilla traffic generators are being used.

4.2.2. FTP (VSFTPD)

I/O graph:

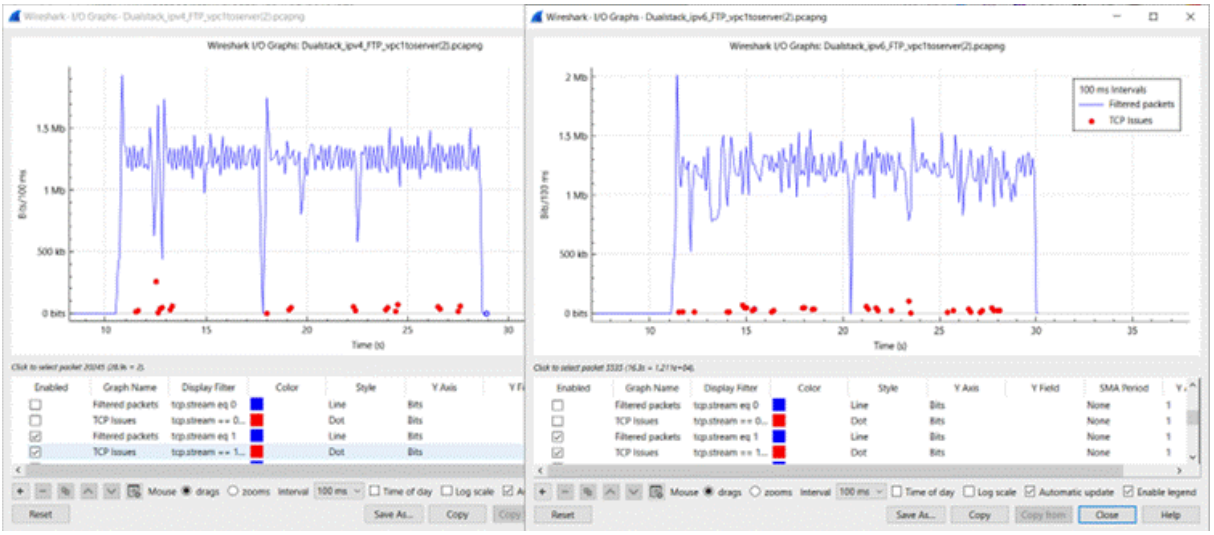


Figure 4.13: I/O graph of FTP WAN capture during the 2nd run

The observed TCP problem counts in the tests of WAN FTP reveal that there is a noticeable run to run variation, however, the general trend is that both IPv4 and IPv6 are similar in their behavior when in the WAN environment. There were 19 TCP problems with IPv4 and 11 with IPv6 in Run 1. This was a different case in Run 2, illustrated in Figure 4.13, where IPv6 scored 32 issues as opposed to IPv4 scoring 21 issues. In run 3 there was hardly any difference, 25 issues on IPv4 and 24 on IPv6. In all three of the runs, IPv4 has a mean of 21.7 issues per test, and IPv6 has 22.3, so each protocol does not always experience more recovery events than the other.

These findings indicate that FTP over the WAN is highly influenced by time variations over short periods as opposed to the decision on the version of IP. The high number of variations in the numbers of issues in the runs indicate that some temporary queueing delays, spikes in the

RTT, or variations in the returns caused brief intervals of retransmissions. This is congruent with the greater variability of RTT in the WAN FTP captures, as both protocols had bursts of delay.

Although the WAN creates more instability compared to the LAN environment, both IPv4 and IPv6 can deal with them. In none of the tests was there any indication of continued congestion, or repeated failure-recovery cycles. The fact that the average of the three runs is virtually the same is a strong indication that the WAN behaviour of FTP is influenced largely by the network conditions at the time of the test and not any underlying performance benefit in either IPv4 or IPv6.

Throughput graph:

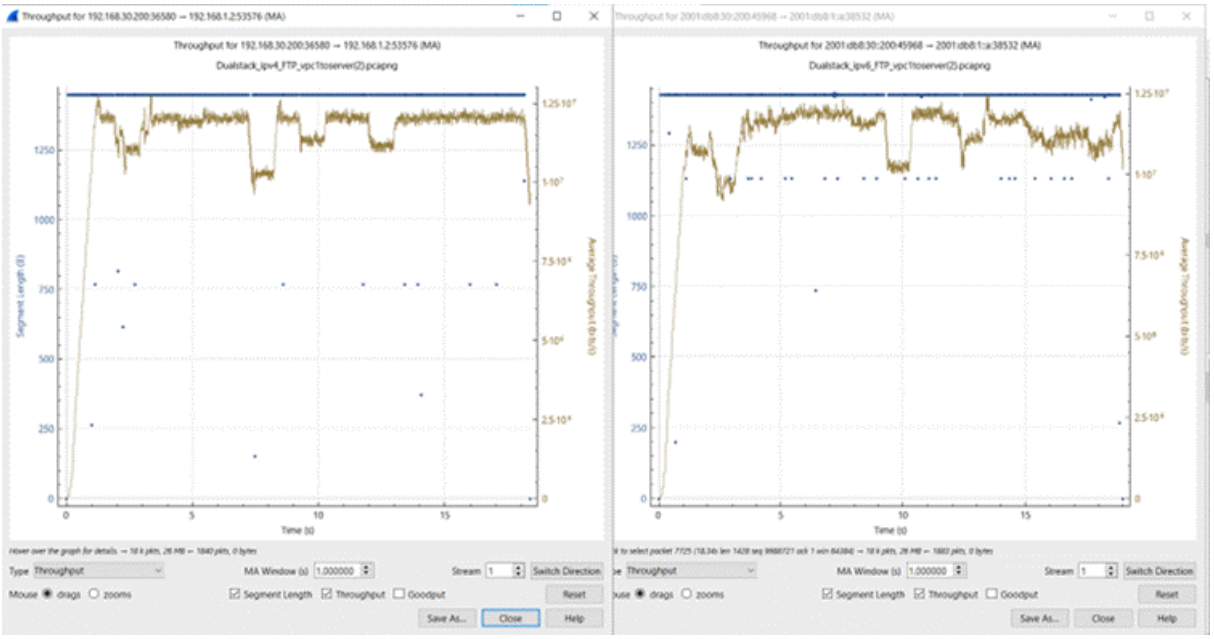


Figure 4.14: *Throughput graph of FTP WAN capture during the 2nd run*

The mean throughput of the three WAN FTP tests reflects the fact that IPv4 and IPv6 provide the performance that is fairly similar to each other, although minor and inconsistent deviations occurred in each run. Run 1 IPv4 had the best throughput of 11.72 Mbps, and IPv6 had 11.52 Mbps, a slim difference of about 1.7 percent. The same trend was also seen in Run 2 as illustrated in Figure 4.14, with IPv4 attaining 12.30 Mbps compared to 11.91 Mbps with IPv6 which is a small margin of about 3.3 Mbps. Nevertheless, the pattern has not continued throughout the experiments. Run 3 had IPv6 slightly ahead of IPv4 at 12.42 Mbps and 12.01 Mbps and an improvement of approximately 3.4%.

In the aggregate outcomes of the three WAN FTPs, the total throughput difference between IPv4 and IPv6 is small and not statistically significant. The two protocols were found to be operating in a range of narrow bandwidth, of about 11.5-12.5 Mbps, a sign of close parity performance on WAN. It is not clear that the IPv4 and IPv6 alternating dominance across runs indicates that variations that are being observed are more of temporary network phenomena, including queuing delays, re-transfers, and application level interactions between FTP and IPv4 and IPv6, as opposed to inherent efficiency differences between the two IP versions.

Round-trip time graph:

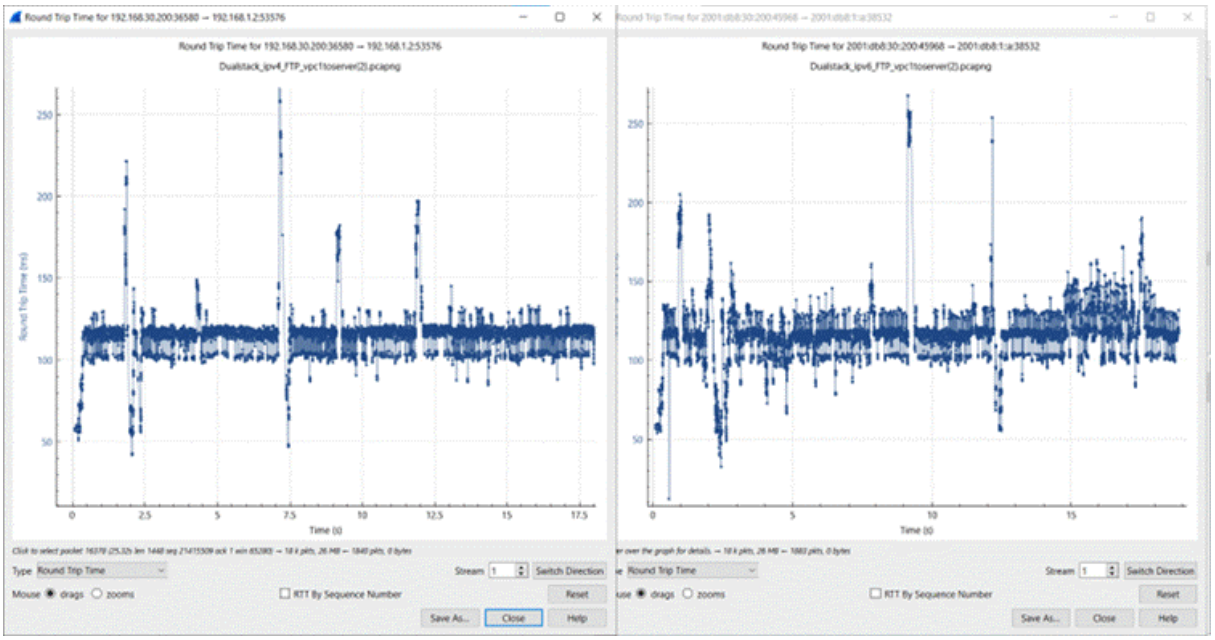


Figure 4.15: *RTT graph of FTP capture during the 2nd run*

The WAN SMB RTT measurements of the three experimental runs reveal that there are only minor differences in the latency behavior in IPv4 and IPv6, and there is also slight variation between the separate runs. Run 1 showed a mean RTT of 0.1118 s of IPv4 and it was slightly higher at 0.1183 s of IPv6. Under run 2 shown in Figure 4.15, both protocols had taken the same value of the RTT, with IPv4 having an average RTT of 0.1224 s, and those of IPv6 having 0.1214 s. The repeated run 3 indicated a variation in the pattern of RTT with the IPv6 getting lower average RTT value of 0.1110 s than 0.1223 s with IPv4.

Looking at all three WAN SMB runs as a batch, both IPv4 and IPv6 exhibit consistent values of RTT between the range of 0.11-0.12 s with no one protocol showing a consistent

domination over the other. Although IPv6 has a slightly superior RTT behavior in some of the runs, the difference is relatively minimal and is not very likely to be noticeable in a real-world implementation. These findings indicate that the effect of network dynamics and interaction effects of protocols are the dominating factors in WAN SMB performance and not inherent RTT benefits of one IP version over the other. Thus, the IPv4 and the IPv6 have similar latency properties of SMB traffic in WAN settings.

4.2.3. SMB (Samba)

I/O graph:

Figure 4.16: *I/O graph of SMB WAN capture during the 3rd run*

The number of TCP problems in the SMB WAN tests is averagely variable though the overall trend is consistent and can be predicted. Run 1 had almost the same results with IPv4 getting 10 TCP issues and IPv6 getting 11. The difference between Run 2 was larger with a single IPv4 having 5 problems and IPv6 having 10. The same pattern was carried on in Run 3, as shown in Figure 4.16, where IPv4 faced 4 problems and IPv6 experienced 10. In all the runs, there are 6.3 IPv4 TCP problems, as compared to 10.3 IPv6 TCP problems, which provides IPv4 with a slight yet steady retransmission advantage.

These increased numbers of issues on IPv6 in Runs 2 and 3 seem to be caused by transient spikes in latency and not sustained congestion. It is consistent with the RTT measurements which indicate that there are some instances of timing spikes that can temporarily interrupt the small-block, high-interaction traffic pattern of SMB. Regardless of this, TCP problems are

relatively small in both protocols, and there is no indication of the presence of persistent congestion, stalled windows, and volatile recovery cycles.

Throughput graph:

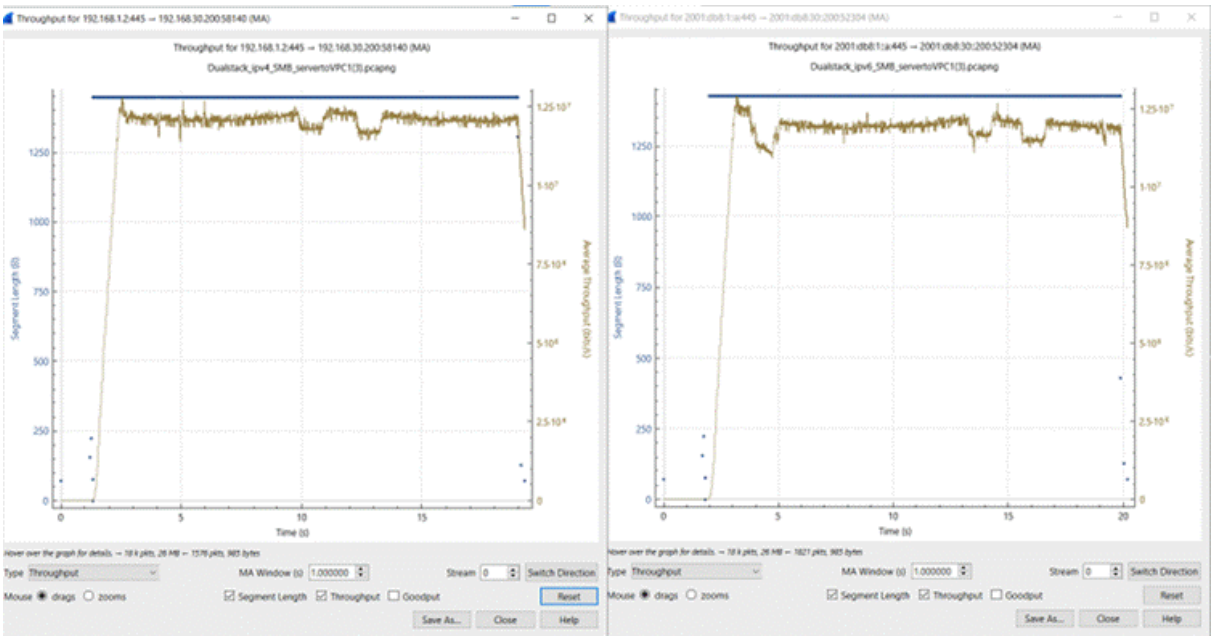


Figure 4.17: *Throughput graph of SMB WAN capture during the 3rd run*

The IPv4 and IPv6 throughput with the three SMB WAN test runs were relatively close with no one protocol demonstrating a stable high throughput rate. During Run 1, IPv6 had a narrow dominance with the mean throughput of 12.21Mbps over IPv4 of 12.09Mbps. The same pattern was followed in run 2 where IPv6 registered 12.57 Mbps and IPv4 registered 12.37 Mbps. Figure 4.17 shows that IPv6 in Run 3 once again recorded slightly better throughput of 12.57 Mbps, whilst IPv4 recorded 12.37 Mbps. An aggregate of all runs shows IPv4 has a mean throughput of 12.27 Mbps and IPv6 has a mean throughput of 12.45 Mbps with an advantage of 0.18 Mbps or about 1.5 percent over IPv4, which falls well within the normal range of variation of WAN.

These findings suggest that IPv4 and IPv6 have a high similarity in terms of SMB throughput over the WAN. The minor variations of every run are just natural variations of path delay and intervening queueing and not a protocol level difference in efficiency. The fact that the value of throughputs remained constant in all the runs also bears out that the request response

behavior of SMB upon the two protocols responds in a similar fashion (there is no evidence of throughput collapse, or protocol specific bottlenecks).

Round-trip time graph:

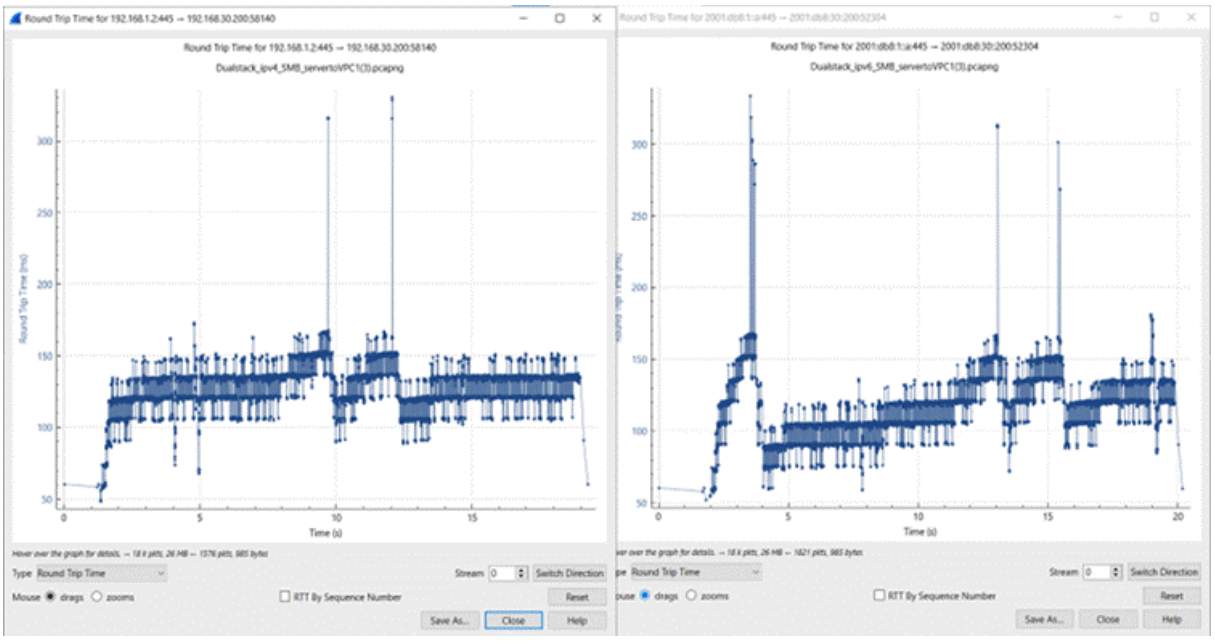


Figure 4.18: *RTT graph of SMB capture during the 3rd run*

There are relatively small variations in RTT behaviour over the SMB WAN tests in IPv4 and IPv6, although, once again, there is no strong long-term benefit of any of the protocols. The average RTT of IPv4 was 0.1118 s in Run 1 and IPv6 recorded a marginally more 0.1183 s. Run 2 recorded almost the same performance where IPv4 had an average of 0.1224 s, and IPv6 had an average of 0.1214 s. Run 3 in Figure 4.18 once more IPv4 averaged 0.1223 s, with IPv6 averaging lower at 0.1110 s, and overall IPv6 has a slight overall average of 1.7 per cent lower RTT.

Although there are a few instances of RTT reared ups particularly during Runs 1 and 2 of both protocols, the overall trend of latency is not changing. The change of IPv4 and IPv6 between the runs implies that delay bursts and not protocol overhead actually condition the differences observed. These WAN RTT variations are normal and have no significant impact on SMB to have a stable data flow.

4.3. Result summary

Table 3: Capture result summary

Topology	**Protocol**	**TCP issues (IPv4 vs. IPv6)**	**Throughput (Mbps) (IPv4 vs. IPv6)**	**RTT (s) (IPv4 vs. IPv6)**	**Analysis**
LAN	iPerf3	16.7 vs. 17	30.14 vs. 38.72	0.0021 vs 0.0022	IPv4 marginally more efficient due to header overhead
LAN	FTP	8.67 vs. 9.33	42.71 vs. 38.73	0.00205 vs. 0.00265	IPv4 benefits from lower protocol overhead
LAN	SMB	8.3 vs. 9.0	42.37 vs. 43.62	0.224 vs. 0.209	IPv6 shows smoother interactive behavior
WAN	iPerf3	6.3 vs. 4.0	11.73 vs. 11.94	0.0440 vs. 0.0340	IPv6 benefits from smoother WAN timing

WAN	FTP	21.7 vs. 22.3	11.73 vs. 12.01	0.11 vs. 0.12	WAN dynamics dominate performance
WAN	SMB	6.3 vs. 10.3	12.27 vs. 12.45	0.1188 vs. 0.1169	Both protocols behave similarly under WAN SMB

The findings indicate that differences in the performance of IPv4 and IPv6 vary greatly according to context and depend on the network scope (LAN vs WAN) and the behavior of applications instead of being related to the inability of the protocol to operate efficiently.

With routing path lengths short and latency low in the LAN environment, IPv4 usually performed marginally better in bulk-transfer workloads, including iPerf3 and FTP. This has been credited to the smaller header size of IPv4 which gives slightly better efficiency to payload when performance is dominated by overhead. These differences were however, small and non-random, suggesting the fact that IPv6 does not pose any significant performance drawdown in local networks where such drawdown is significant.

In interactive load such as SMB, IPv6 usually showed the same or even better throughput and RTT stability. This is probably due to the fixed size IPv6 header and the simplified forwarding behavior being more predictable and beneficial to applications which are sensitive to frequent request-response interactions. Notably, TCP stability indicators in the LAN exhibited insignificant variations, which proved that IPv6 does not impact adversely on congestion control and recovery behavior.

Conversely, the WAN environment emphasized various performance features. Header costs were very much negligible as compared to routing delays, queueing influences, and multi-hop handling. It was observed in this respect that IPv6 often had lower average RTT and smoother

delay distributions, especially in iPerf3 and FTP tests. These gains were in the form of slightly better and more predictable throughput, particularly when sustained TCP flows were concerned.

The counts of TCP issues during WAN tests were more variable on the whole, which indicates the dynamism of the WAN conditions. But both protocols did not show sustained instability and the variation in retransmission or recovery events were small. Where variations were determined, they were usually in line with RTT spikes and not protocol-related behavior.

In all situations, one of the major discoveries is that IPv6 performance is at minimum equal with IPv4 and is slightly better in various WAN situations. There was no test with systematic degradation due to IPv6. Rather, the findings corroborate findings of recent literature that IPv6 is competitive when it is implemented in native well-configured settings.

CHAPTER 5

DISCUSSION AND RECOMMENDATION

5.1. Improvement recommendation

5.1.1. Favouring Native IPv6 connectivity

When feasible, Native IPv6 connection must be favoured compared with transitional facilities, since tunnelling and translation methods add processing overheads, and complicate paths. The transition technologies (IPv6-over-IPv4 tunnels) add additional headers to packets, minimizing the effective Maximum Transmission Unit (MTU) and increasing the risk of fragmentation-related problems and longer latency [21]. Research and vendor documentation has shown that the IPv6 traffic tunneling can take suboptimal or asymmetric routes, which results in an increase in the RTT variability and a decrease in the stability of throughput [22]. Contrarily, native IPv6 paths avoid encapsulation overhead and allow packets to be carried out without a first-level forwarding through IPv6-aware routing infrastructure, leading to a more foreseeable performance. The IPv6 implementation recommendations at Cisco further affirm the fact that a dual-stack architecture is always more efficient in throughput and latency predictability than a tunneled application whenever hardware support is made [23].

5.1.2. Routing Symmetry and Traffic Engineering

Routing design and traffic engineering practices have a significant effect on IPv6 performance. Routing policies in IPv6 networks are not as developed as their IPv4 counterparts and thus may result in longer routes or asymmetric routes that raise RTT and reordering of packets. Studies of IPv6 deployment practice indicate that the intermittency of performance due to variations in peering policies and prefix advertisement policies is common but protocol inefficiencies are rare [9][10]. Symmetric routing of IPv6 traffic ensured assists in the stabilization of RTT as well as enhanced TCP congestion window expansion especially within a WAN setup. The complex routing tables are mitigated by the current routing methods like careful aggregation of prefixes and consistent policy implementation which also lead to convergence behaviour and subsequently an increase in throughput stability. According to industry recommendations, IPv6 traffic control must receive the same degree of rigour as IPv4 to prevent the occurrence of unwanted performance differences [23].

5.1.3. Hardware Capability / Software Stack Maturity

IPv6 forwarding performance is directly related to the performance of network hardware and network stacks in operating systems. Prior generations of routers and switches tended to handle IPv6 traffic using software instead of hardware, providing reduced throughput and increased latency than IPv4 could. However, modern networking equipment generally provides complete hardware support to perform IPv6 forwarding, access control and quality of service, enabling IPv6 traffic to be handled at line rate [21]. On the same note, modern operating systems use optimised IPv6 TCP/IP stacks that enhance better buffer management, congestion and PMTUD support. Regularly updating old infrastructure and ensuring the use of current software are the recommended practices of industry deployment that would help to align the IPv6 performance with the IPv4 expectations [23].

5.1.4. Correct DNS Configuration and Name Resolution Optimization

DNS behavior plays a critical role in perceived IPv6 performance and is often overlooked during deployment. Incorrect or inconsistent DNS configuration can result in connection delays, failed lookups, or suboptimal protocol selection, even when the underlying IPv6 network is functioning correctly. To avoid these issues, DNS servers must be properly configured to support both A and AAAA records and respond efficiently to IPv6 queries. Ensuring that DNS records are accurate and synchronized prevents fallback delays caused by unreachable IPv6 endpoints. Furthermore, deploying DNSSEC improves the integrity and reliability of name resolution, which is particularly important in IPv6 networks where address space size increases the importance of automated and trustworthy DNS operations.

5.1.5. Firewall, Security Policy, and Software Readiness

Security infrastructure must be fully IPv6-aware to prevent unintended performance degradation. Firewalls and intrusion detection systems that rely on IPv4-centric inspection models may introduce latency when processing IPv6 packets or inadvertently block essential control traffic such as ICMPv6, which is critical for PMTUD and neighbor discovery. IPv6-compatible firewalls should be configured with policies tailored specifically for IPv6 traffic rather than mirroring IPv4 rulesets. Keeping firewall firmware and security software up to date ensures that performance optimizations and protocol enhancements are supported,

while also reducing the risk of vulnerabilities that could indirectly impact network stability and throughput.

5.1.6. Ongoing Observation and Performance Checking

Optimization of the performance of IPv6 must be considered as continuous and not a one-time configuration exercise. Constant checking of throughput, RTT, packet loss and TCP behaviour allows detection of routing modifications, configuration mistakes or infrastructure constraints that in turn can unproportionately impact the IPv6 traffic. Operational guidelines and measurement platforms also stress that it is important to develop baseline performance metrics and compare IPv4 and IPv6 behaviour with time in order to achieve parity [10][11]. Active monitoring enables network operators to justify optimisation procedures and ensure that the service quality remains unchanged with the development of the network conditions.

5.1.7. Optimization of Applications

The experimental findings of this thesis prove that the features of applications strongly influence the appearance of the IPv6 performance differences. Applications in which sustained throughput and congestion window dynamics are the most relevant include bulk-transfer applications like iPerf3 and FTP, whereas applications that are interactive such as SMB are more sensitive to RTT stability and packet pacing. Traffic patterns and protocol behaviour Application-aware optimization strategies can thus be used to enhance overall IPv6 performance. The industry advice recommends optimisation of queue management, buffer placement and traffic prioritisation to match application needs to improve user experience without adding bias in the protocols [23].

5.2. OSPFv3 cost implementation and optimization

5.2.1. Multipath routing and its application in real-life

Multipath routing is a networking strategy that is extensively used in modern service provider and enterprise networks and allows multiple routing paths to exist between a source and a destination. Its main aims are to enhance the network availability, increase fault tolerance, and provide traffic engineering requirements. Networks are able to maintain operations during link or node failures, by having alternate paths available, and networks are also enabling flexible traffic management policies. Multi path routing Multipath routing is one of the

foundations of real world network design and technologies like OSPF, IS-IS, BGP and MPLS are designed around the concept of multipath routing design.

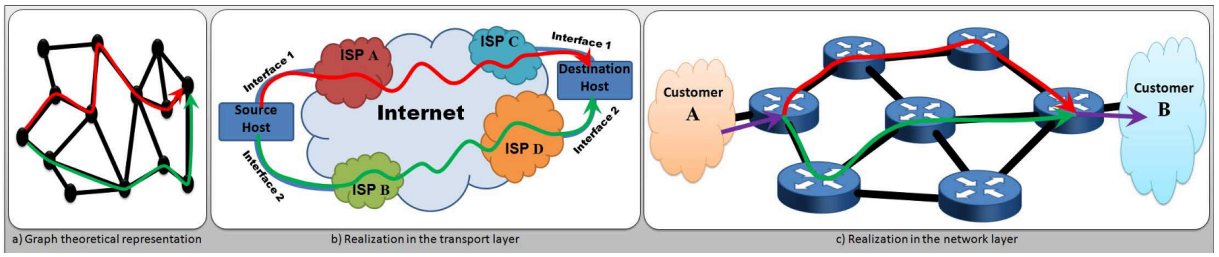


Figure 5.1: *Multipath routing illustration*

Figure 3.1 shows the network topology, including 1 Local area network (LAN) connection between VM1 and VM2 inside a same subnet with Router 1 (R1) as default gateway, 2 Wide area network (WAN) connections between VM1, VM2 and server with different subnets, separating by 2 routers.

This topology also includes a Network address translation (NAT) for IPv4 connection to reach the public network

In practice, multipath routing is not necessarily aimed at load balancing. To the contrary, the differentiation between a primary and a secondary path is made by many production networks to guarantee predictable forwarding behavior. Backup routes are usually set to have higher routing metrics to ensure that they are available to use in case of a disaster without interrupting the normal traffic flow. The design decisions mirror operational priorities which include consistency of operation through stability, determinism, and consistency of performance, which is of particular concern to the latency-sensitive and high-throughput applications. Therefore, routing schemes are based on metric or cost schemes to implement routing preferences among the various possible paths.

OSPFv3 is highly important in these multipath designs in IPv6 networks by selecting routes based on costs. Although default OSPFv3 cost computations may offer rudimentary connectivity along with connectivity, they might not always indicate the planned routing policy or performance features in an environment with a number of equally good paths. This is the reason for the explicit OSPFv3 cost optimization, with the routing metrics being adjusted to guarantee that the traffic is conducted along the most efficient paths in a typical situation, yet the alternative paths are required to be used in the redundancy situation. This

paper takes a simplified multipath topology to investigate the effect of OSPFv3 cost setting on the IPv6 routing practice and network performance.

5.2.2. OSPFv3 Overview in IPv6 routing

Open Shortest Path First version 3 (OSPFv3) is a link-state routing protocol specifically designed to support IPv6 networks. Unlike OSPFv2, which is tightly coupled with IPv4 addressing, OSPFv3 separates routing logic from IP addressing and operates directly over IPv6 using link-local addresses for neighbor adjacency formation. This design improves scalability and flexibility in IPv6 environments, particularly in multi-interface and multi-address scenarios.

In OSPFv3, routers exchange Link-State Advertisements (LSAs) to construct a complete view of the network topology. Based on this information, each router independently computes the shortest path tree using Dijkstra's algorithm. As a result, routing decisions are made consistently across the network, enabling predictable and loop-free forwarding behavior.

5.2.3. Cost Metric and Path Selection Mechanism

OSPFv3 determines the preferred routing path based on the cumulative cost of a route rather than hop count. The cost is assigned on a per-interface basis and represents the relative expense of forwarding traffic through that interface. During route computation, OSPFv3 calculates the total path cost by summing the costs of all outgoing interfaces along a route. The path with the lowest cumulative cost is selected as the best route.

By default, OSPF cost is derived from interface bandwidth; however, this automatic calculation often fails to reflect real performance characteristics in virtualized or laboratory environments. Consequently, manual cost tuning is commonly employed in both research and production networks to better control routing behavior. The numerical values of cost are not significant on their own; rather, the relative difference between alternative paths determines route preference.

5.2.4. Equal-Cost Multipath (ECMP) and Its Implications

When multiple routes to the same destination have identical cumulative cost, OSPFv3 installs all eligible routes into the routing table and performs Equal-Cost Multipath (ECMP)

forwarding. In this mode, traffic is distributed across multiple paths on a per-flow basis, typically using a hashing mechanism.

Although ECMP can improve bandwidth utilization, it introduces variability in packet delay and throughput, which may negatively affect performance evaluation. For experimental studies focusing on routing efficiency and stability, ECMP can obscure the impact of optimization techniques by introducing non-deterministic traffic distribution. Therefore, ECMP behavior must be carefully controlled or avoided when consistent performance measurement is required.

5.2.5. Cost-Based Routing Optimization Strategy

Routing optimization in OSPFv3 can be achieved by manually adjusting interface cost values to influence path selection. By assigning lower costs to preferred links and higher costs to less efficient or longer paths, network operators can enforce deterministic routing behavior without modifying the physical topology.

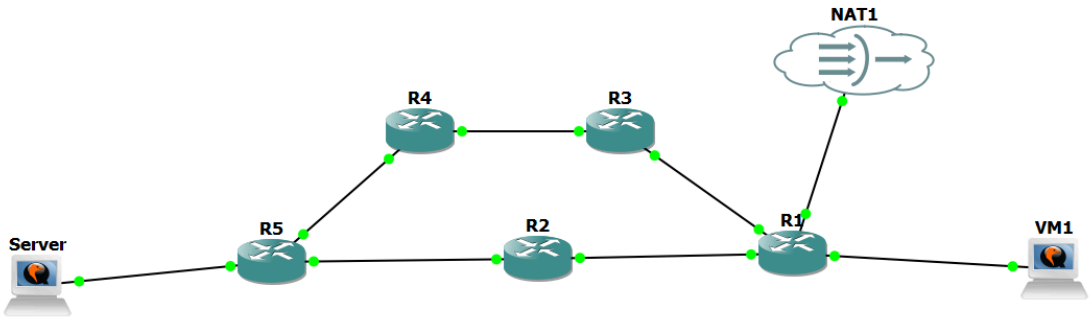


Figure 5.2: *OSPFv3 IPv6 experimenting network topology*

There are two different routing paths between the Server and R1 in the topology in Figure 5.2. The primary path goes through R1 - R2 - R5 - Server and the other alternative path is through R1 - R3 - R4 - R5 - Server. But despite the fact that both paths are fully connected, the second one adds more hops and processing delays, which might harm latency-sensitive and throughput-dependent applications in case they are chosen accidentally.

Shared cost values were set to achieve predictable and constant routing behavior. A cost value of 10 per interface was allocated to all interfaces on the R2-R5 primary route giving a reduced cumulative path cost. Interfaces on the R3-R4-R5 reserved route, in contrast, had a higher

cost value of 50 per interface, intentionally adding to the total cost of this route. Consequently, in a stable operation environment, OSPFv3 always picks the R2-R5 path as the preferable forwarding path whereas the R3-R4-R5 route is only used as a backup option.

This optimization strategy is based on the cost-driven routing in the real world and the primary paths are currently designed to be the most efficient and secondary paths are intact to provide resilience and failover instead of load sharing. Using metric tuned separation of operational functions, the network becomes more stable in routing, and does not experience unintended ECMP behavior, and the results of performance evaluation correctly represent the properties of the desired primary path.

5.2.6. Throughput result comparison

iPerf3:

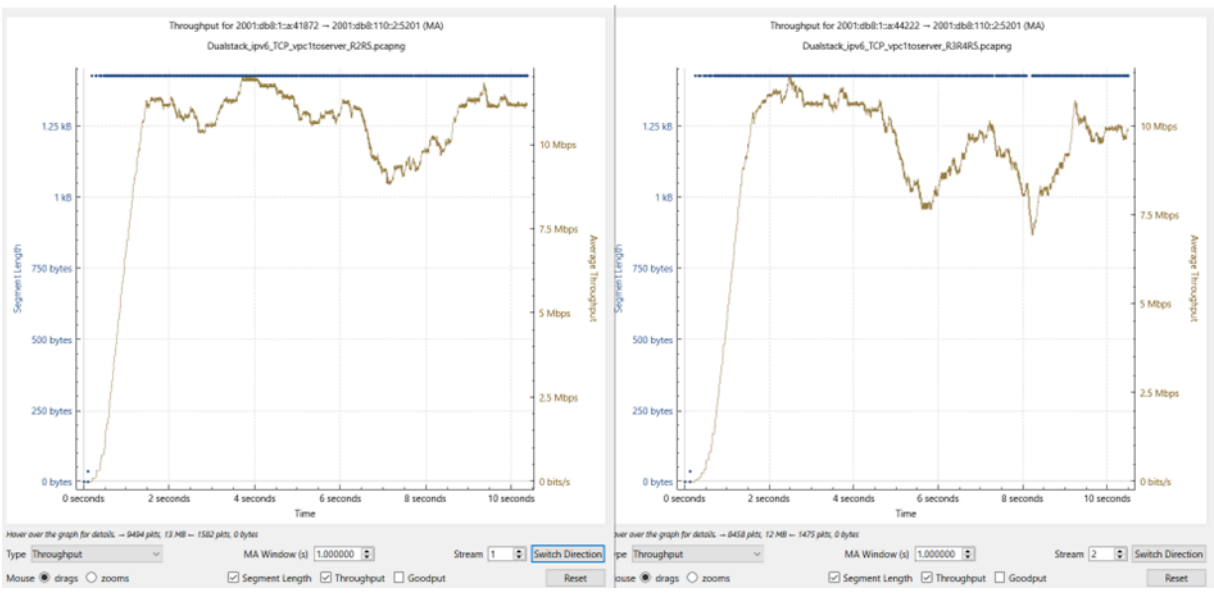


Figure 5.3: *Throughput performance of R2 - R5 vs. R3 - R4 - R5 routes in iPerf3*

The results of the iPerf3 throughput illustrated in Figure 5.3 indicate the optimized routing path over R2 - R5 has a steady performance advantage over the longer one over R3 - R4 - R5. The result shows that the R2 - R5 route had an average throughput of 11.15 Mbps compared to the lower average of 9.40 Mbps of the R3 - R4 - R5 path as seen in the figure. This is a significant decrease in throughput as the traffic crosses the extra routing hops.

On the whole, the iPerf3 findings support the argument that the optimisation of routing cost-based in OSPFv3 effectively directs IPv6 traffic over the more efficient path, which leads to the increase in average throughput and better the stability of the performance.

FTP:

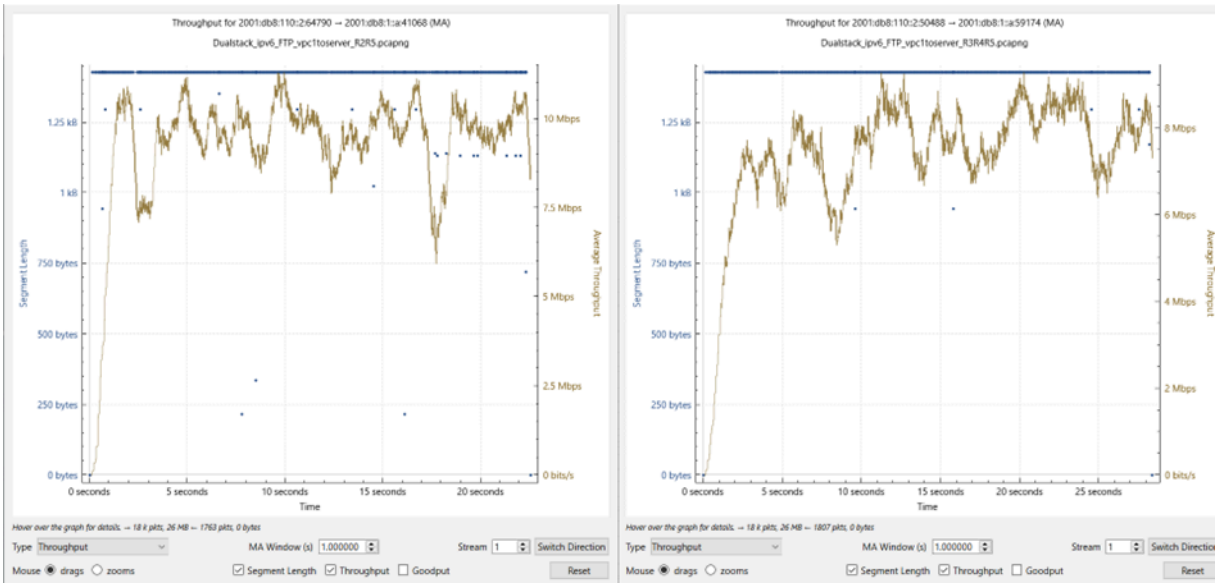


Figure 5.4: *Throughput performance of R2 - R5 vs. R3 - R4 - R5 routes in FTP*

The performance of the FTP throughput in Figure 5.4 indicates further the network gap in the performance of the optimized and non-optimized routing paths. With traffic bypassing R2 - R5 the throughput was 10.09 Mbps and the longer path of R3 - R4 - R5 had a throughput of only 8.19 Mbps. This decrease is more than that in the iPerf3 experiments, or it is indicative of the fact that FTP is more sensitive to routing inefficiencies.

These findings are indicative of the fact that more routing complexity and hops have more impact on FTP traffic, which entails both data and control channel interaction. As a result, the routing optimization becomes an important issue in ensuring FTP performance that is stable and efficient over IPv6 networks.

SMB:

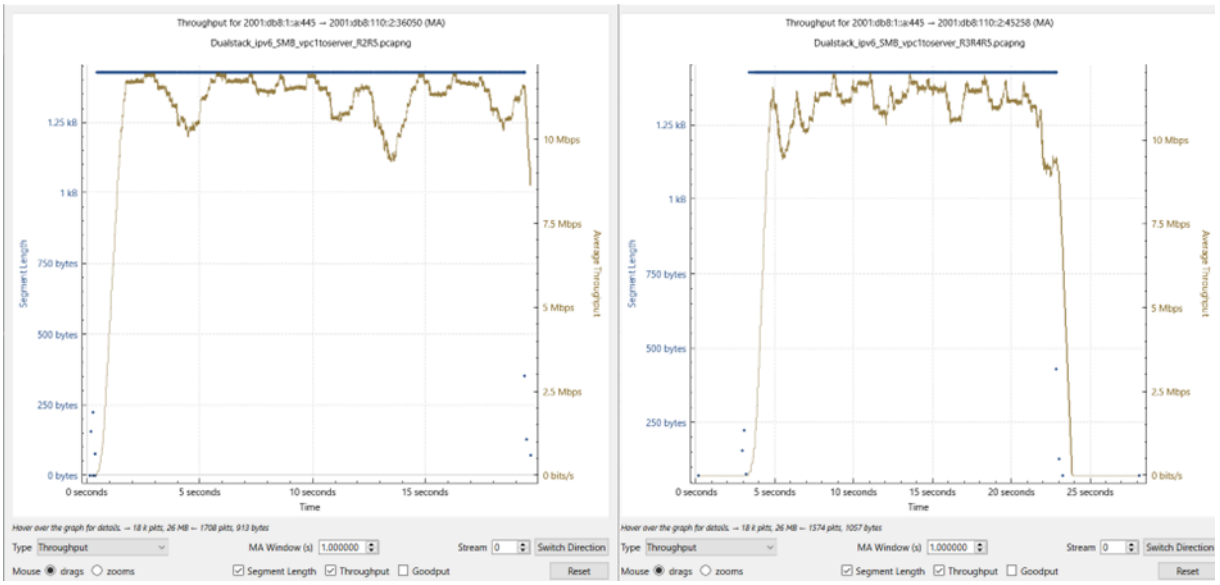


Figure 5.5: *Throughput performance of R2 - R5 vs. R3 - R4 - R5 routes in SMB*

Figure 5.5 shows that the SMB throughput results have smaller, although still consistent, benefits of the optimized R2 - R5 routing path. The mean throughput between R2 - R5 was 12.02 Mbps which was a little better than that of the route between R3 - R4 - R5 where the mean was 11.67 Mbps. The difference is not as high as compared to iPerf3 and FTP, however, it can be evaluated and measured throughout the test period and across all samples.

All in all, the SMB results suggest that throughput consistency is the main benefit of routing optimization, not peak performance, as it is very crucial in file-sharing workloads, which are based on consistent rates of data transfer.

5.3. Path MTU mismatch and Path MTU Discovery

5.3.1. Maximum Transfer Unit and Path MTU

The Maximum Transfer Unit (MTU) refers to the packet size which cannot be sent on a network interface without being fragmented. MTU is a very important concept in the communication at the end to end of an IPv6 network since the routers mid-way are not permitted to fragment packets. Rather, the sender manages the use of packet fragmentation purely on the basis of the Path MTU (PMTU) of the path between source and destination.

Path MTU is used to denote the minimum value of all links in a forwarding path. In order to find this value, IPv6 uses Path MTU Discovery (PMTUD) which uses ICMPv6 Packet Too Big messages to notify the sender that the packets it is sending are larger than the maximum

supported MTU of an intermediate connection. When such feedback is received, the sender increases or decreases the size of its packets. The mechanism is efficient in delivering packets without fragmentation in the network.

Nevertheless, the functionality of PMTUD requires proper signaling of ICMPv6 and proper management at the transmitter. Delays or blockage of PMTU information can cause discrepancies between the sender MTU and the actual path MTU resulting in the loss of packets, retransmissions and poor performance of the applications. Thus, to optimize the performance of IPv6, it is necessary to know the work of MTU and the working of PMTUD.

5.3.2. Path MTU mismatch

A scenario of Path MTU mismatch was deliberately introduced during this experiment to test IPv6 behavior in a non-optimal scenario with regards to MTU. The sender had an MTU set to 1500 bytes and a router halfway down the forwarding path had a lower MTU of 1400 bytes. Such a setup led to the sender sending packets bigger than the size that could be allowed in the path.

According to the captured traffic, the intermediate router was producing various ICMPv6 Packet Too Big packets, which meant that packets bigger than the maximum MTCU could not be sent. At this stage, retransmission of TCP packets and out of order packets were detected. These symptoms allude to the fact that the sender still sent oversized packets before it successfully changed its transmission behavior.

Such a mismatch had an adverse impact on the network behavior as it added the overhead of retransmission and slowed down the data delivery. Applications that used TCP like FTP suffered throughput reduction as a result of re-transmission of packets and congestion wind-down. The given situation in Figure 5.6 shows that an even rather minor difference in MTUs can create a severe effect on the IPv6 communications in case the PMTU information is not fully implemented by the sender yet.

23	3.690252	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
24	3.690252	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
25	3.690252	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
26	3.690252	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
27	3.690252	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
28	3.690252	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
29	3.690708	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
30	3.690708	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
31	3.690708	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
32	3.690708	2001:db8:30::1	2001:db8:30::200	ICMPv6	1294	Packet Too Big
33	3.691738	2001:db8:30::200	2001:db8:1::a	TCP	1414	[TCP Retransmission] 49015 → 46168 [ACK] Seq=1 Ack=1 Win=64384 Len=1328 TSval=2892681420 TSecr=1364426986
34	3.692194	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	[TCP Out-Of-Order] FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
35	3.692194	2001:db8:30::200	2001:db8:1::a	TCP	286	[TCP Retransmission] 49015 → 46168 [ACK] Seq=2657 Ack=1 Win=64384 Len=200 TSval=2892681420 TSecr=1364426986
36	3.692194	2001:db8:30::200	2001:db8:1::a	TCP	1414	[TCP Retransmission] 49015 → 46168 [ACK] Seq=2857 Ack=1 Win=64384 Len=1328 TSval=2892681420 TSecr=1364426986
37	3.692194	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	[TCP Out-Of-Order] FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
38	3.692690	2001:db8:30::200	2001:db8:1::a	TCP	286	[TCP Retransmission] 49015 → 46168 [ACK] Seq=5513 Ack=1 Win=64384 Len=200 TSval=2892681420 TSecr=1364426986
39	3.693186	2001:db8:30::200	2001:db8:1::a	TCP	1414	[TCP Retransmission] 49015 → 46168 [ACK] Seq=5713 Ack=1 Win=64384 Len=1328 TSval=2892681420 TSecr=1364426986
40	3.693186	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	[TCP Out-Of-Order] FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
41	3.693683	2001:db8:30::200	2001:db8:1::a	TCP	286	[TCP Retransmission] 49015 → 46168 [ACK] Seq=8369 Ack=1 Win=64384 Len=200 TSval=2892681434 TSecr=1364426986
42	3.693683	2001:db8:30::200	2001:db8:1::a	TCP	1414	[TCP Retransmission] 49015 → 46168 [ACK] Seq=8569 Ack=1 Win=64384 Len=1328 TSval=2892681434 TSecr=1364426986

Figure 5.6: *ICMPv6 "Packet too big" messages*

5.3.3. Path MTU Discovery in action

Once Path MTU mismatch had been identified, the Path MTU Discovery mechanism became functional as desired. On getting an ICMPv6 Packet Too Big messages by the middle router, the sender decreased its effective MTU by changing 1500 bytes to 1400 bytes, which matched the minimum MTU on the path.

After this adjustment, there is a significant decrease in retrasmission and the stable transfer of TCP data is achieved, as shown in Figure 5.7. The size of packets were decreased accordingly, and the further transmissions were managed to be passed and never again fragmented without errors connected to fragmentation. This affirmation confirms that PMTUD is an effective mechanism in solving mismatch of the MTU after a given amount of feedback is obtained by the sender.

The behavior observed demonstrates the reactive characteristic of PMTUD in IPv6 networks. Although PMTUD can effectively recover performance in communication when it has identified a constraint in the MTU, there is still the possibility of temporary deterioration of performance in the initial mismatch period. This further makes the difference of consistency of the MTU and correct functioning of PMTUD in performance-sensitive IPv6 deployments important.

38	3.692690	2001:db8:30::200	2001:db8:1::a	TCP	286	[TCP Retransmission] 49015 → 46168 [ACK] Seq=5513 Ack=1 Win=64384 Len=200 TSval=2892681420 TSecr=1364426986
39	3.693186	2001:db8:30::200	2001:db8:1::a	TCP	1414	[TCP Retransmission] 49015 → 46168 [ACK] Seq=5713 Ack=1 Win=64384 Len=1328 TSval=2892681420 TSecr=1364426986
40	3.693186	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	[TCP Out-Of-Order] FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
41	3.693683	2001:db8:30::200	2001:db8:1::a	TCP	286	[TCP Retransmission] 49015 → 46168 [ACK] Seq=8369 Ack=1 Win=64384 Len=200 TSval=2892681434 TSecr=1364426986
42	3.693683	2001:db8:30::200	2001:db8:1::a	TCP	1414	[TCP Retransmission] 49015 → 46168 [ACK] Seq=8569 Ack=1 Win=64384 Len=1328 TSval=2892681434 TSecr=1364426986
44	3.751716	2001:db8:1::a	2001:db8:30::200	TCP	86	46168 → 49015 [ACK] Seq=1 Ack=1329 Win=133896 Len=0 TSval=1364427062 TSecr=2892681420
45	3.751716	2001:db8:1::a	2001:db8:30::200	TCP	86	46168 → 49015 [ACK] Seq=1 Ack=2657 Win=136752 Len=0 TSval=1364427064 TSecr=2892681420
46	3.752210	2001:db8:1::a	2001:db8:30::200	TCP	86	46168 → 49015 [ACK] Seq=1 Ack=2857 Win=136552 Len=0 TSval=1364427064 TSecr=2892681420
47	3.752210	2001:db8:1::a	2001:db8:30::200	TCP	86	46168 → 49015 [ACK] Seq=1 Ack=5513 Win=134676 Len=0 TSval=1364427064 TSecr=2892681420
48	3.752210	2001:db8:1::a	2001:db8:30::200	TCP	86	46168 → 49015 [ACK] Seq=1 Ack=5713 Win=134476 Len=0 TSval=1364427064 TSecr=2892681420
49	3.752210	2001:db8:1::a	2001:db8:30::200	TCP	86	46168 → 49015 [ACK] Seq=1 Ack=8369 Win=132228 Len=0 TSval=1364427064 TSecr=2892681420
50	3.752210	2001:db8:1::a	2001:db8:30::200	TCP	86	46168 → 49015 [ACK] Seq=1 Ack=8569 Win=132028 Len=0 TSval=1364427064 TSecr=2892681434
51	3.752210	2001:db8:1::a	2001:db8:30::200	TCP	86	46168 → 49015 [ACK] Seq=1 Ack=9897 Win=130700 Len=0 TSval=1364427064 TSecr=2892681434
52	3.753700	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	[TCP Out-Of-Order] FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
53	3.753700	2001:db8:30::200	2001:db8:1::a	TCP	286	[TCP Retransmission] 49015 → 46168 [PSH, ACK] Seq=11225 Ack=1 Win=64384 Len=200 TSval=2892681494 TSecr=1364427062
54	3.753700	2001:db8:30::200	2001:db8:1::a	TCP	1414	[TCP Retransmission] 49015 → 46168 [ACK] Seq=11425 Ack=1 Win=64384 Len=1328 TSval=2892681494 TSecr=1364427064
55	3.754195	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	[TCP Out-Of-Order] FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
56	3.754195	2001:db8:30::200	2001:db8:1::a	TCP	286	[TCP Retransmission] 49015 → 46168 [ACK] Seq=14081 Ack=1 Win=64384 Len=200 TSval=2892681494 TSecr=1364427064
57	3.754692	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
58	3.754692	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
59	3.754692	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
60	3.754692	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
61	3.755186	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
62	3.755186	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
63	3.755186	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
64	3.755682	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
65	3.755682	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)
66	3.755682	2001:db8:30::200	2001:db8:1::a	FTP-DA…	1414	FTP Data: 1328 bytes (EPASV) (RETR python-3.12.0-amd64.exe)

Figure 5.7: *PMTUD adjust the MTU of sender*

CHAPTER 6

DISCUSSION, CONCLUSION AND FUTURE WORK

6.1. Discussion

The findings of the given study prove that the IPv6 performance is very context-dependent and must not be assessed without references to network coverage, routing environment, and application characteristics. In both the LAN and WAN settings, IPv6 was never found to have any intrinsic performance drawbacks as compared to IPv4. Rather, the differences that were observed were greatly due to operational parameters like the length of the routing path, latency characteristics and application workload nature.

In LAN settings, where routing paths are minimal and latency very low, IPv4 was demonstrated to have slightly better throughput in bulk data transfer programs iPerf3 and FTP. This can be explained by the fact that the smaller IPv4 header size causes slightly greater payload efficiency when the protocol overhead is large. Nevertheless, the differences were little, consistent, and not random, which shows that there are no performance penalties due to IPv6 in well-controlled local networks.

With interactive and transaction-based workloads, including SMB, IPv6 showed similar or better behavior in the throughput stability and also consistency in RTT. This is an indication that the fixed-length header provided by IPv6 and its simplified forwarding mechanism could offer applications that are rather sensitive in the number of request-response transactions a more predictable behavior. Notably, the stability indicators of TCP such as retransmissions, duplicate acknowledgments, and out-of-order packets did not indicate any negative difference which could be related to IPv6 use in the LAN environment.

When using WAN, the overheads in protocol header was negligible compared to routing delay, queuing effects, and multi-hop traversing. In this case, IPv6 tended to have better average RTT and less jagged delay and distributions, especially in sustained TCP traffic. Such properties led to the more predictable behavior in throughput in a number of test cases. The two protocols had their variations in the TCP-related events which indicated that the WAN

environments are dynamic, but these variations were associated with RTT changes as opposed to protocol-intransient inefficiency.

In general, the discussion supports the recent literature that IPv6 can match or even be even faster in terms of performance in case it is implemented natively and settings are right. The findings take a strong hint that the performance issues related to IPv6 are more reliant on the maturity of the deployment and network engineering than on the restrictions of the protocol.

The findings in Chapter 5 demonstrate that routing behavior and Path MTU handling are the key factors that determine IPv6 performance, as opposed to protocol inefficiencies. The comparison between the R2-R5 and R3-R4-R5 routes shows that OSPFv3 always prefers the routes with lower cumulative cost which leads to the lower RTT and more constant throughput on shorter routes. Besides, Path MTU Discovery is also important in ensuring the IPv6 is functioning effectively, especially on the multi-hop WAN path, wherein the mismatch of the MTU may raise temporary retransmission and delay. The findings indicate that proper implementation of IPv6 relies greatly on proper cost design of OSPFv3 and continuous ICMPv6 signalling.

6.2. Conclusion

The results show that IPv6 is at least as effective as IPv4 in most of the situations, and even better in WAN systems where routing efficiency and delay properties are more important than protocol overhead. There was no case of performance degradation that could be attributed to IPv6. Rather, context-specific performance variation was displayed to be achievement of the network topology, routing behavior and application characteristics.

These findings add up to the already growing body of empirical evidence to counter the myth that IPv6 is sluggish or just less reliable than IPv4. To network operators and organizations not willing to implement IPv6 because of the perceived performance issues, this study indicates that they are however unfounded when the IPv6 is implemented in pure, well-designed environments.

To sum up, IPv6 is not a performance threat but a space-saving and cost-effective protocol, which can meet current and subsequent Internet demands, assuming that appropriate deployment and optimization guidelines are used.

Moreover, OSPFv3 routing on a cost basis enables the selection of the deterministic path to be used, so that in normal operation the backup routing via long paths is not used, yet the network is resilient. Simultaneously, the PMTUD allows IPv6 to dynamically extend to the constraints of the MTU without using in-network fragmentation. In general, the findings suggest that the performance of IPv6 is mostly configuration-related, but not protocol-related and that routing optimization and the consistency of the MTU in IPv6 networks are paramount.

6.3. Future work

Although this research has presented great information about IPv4 and IPv6 performance, there are still a number of places that can be explored. To begin with, future studies might take the experimental configuration further in order to cover more complicated and extensive network structures adding several intermediary routers and various routing protocols to have an improved reflection of the real-life ISP settings.

Second, more performance indicators like packet loss, jitter, CPU usage, and energy use could be also examined and supported to present a more holistic design of protocol efficiency that is especially suitable in latency sensitive and real-time performance. The analysis of such measures would be particularly applicable to the new cases of applications like the Internet of Things, cloud computing, and streaming of multimedia based on IPv6.

Lastly, it is possible to investigate in future methods to evaluate the difference in performance between dual-stack, NAT64, and tunneling as the techniques in future studies using transition mechanisms since many production networks are still based on a hybrid IPv4/IPv6 configuration. A comparison between the native IPv6 and transitional approaches would present feasible advice to organizations at intermediate stages of migration.

REFERENCES

1. "Internet Protocol," RFC 791, 1981, https://tools.ietf.org/html/rfc791.

2. S. Deering and R. Hinden, "Internet Protocol version 6 (IPv6)," RFC 2460, 1998, https://www.ietf.org/rfc/rfc2460.

3. F. Li, C. An, J. Yang, J. Wu, and H. Zhang, "A study of traffic from the perspective of a large pure IPv6 ISP," Computer Communications, vol. 37, pp. 40–52, 2014.

4. F. Li, J. Yang, J. Wu, Z. Zheng, H. Zhang, and X. Wang, "Configuration analysis and recommendation: Case studies in IPv6 networks," Computer Communications, vol. 53, pp. 37–51, 2014.

5. F. Li, J. Yang, X. Wang, T. Pan, C. An, and J. Wu, "Characteristics analysis at prefix granularity: A case study in an IPv6 network," Journal of Network and Computer Applications, vol. 70, pp. 156–170, 2016.

6. Y. Wang, S. Ye, and X. Li, "Understanding current IPv6 performance: A measurement study," in Proceedings of the 10th IEEE Symposium on Computers and Communications, ISCC 2005, pp. 71–76, esp, June 2005.

7. X. Zhou, M. Jacobsson, H. Uijterwaal, and P. Van Mieghem, "IPv6 delay and loss performance evolution," International Journal of Communication Systems, vol. 21, no. 6, pp. 643–663, 2008.

8. F. Li, X. Wang, T. Pan, and J. Yang, "Packet delay, loss and reordering in IPv6 world: a case study," in Proceedings of the International Conference on Computing, Networking and Communications (ICNC '16), pp. 1–6, February 2016.

9. A. Dhamdhere, M. Luckie, B. Huffaker, K. Claffy, A. Elmokashfi, and E. Aben, "Measuring the deployment of IPv6: topology, routing and performance," in Proceedings of the ACM Internet Measurement Conference (IMC '12), pp. 537–550, Boston, Mass, USA, November 2012.

10. J. Czyz, M. Allman, J. Zhang, S. Iekel-Johnson, E. Osterweil, and M. Bailey, “Measuring IPv6 Adoption,” in ACM SIGCOMM Computer Communication Review, vol. 44, pp. 87–98, 2014.

11. I. Livadariu, A. Elmokashfi, and A. Dhamdhere, “Characterizing IPv6 control and data plane stability,” in Proceedings of the 35th Annual IEEE International Conference on Computer Communications, IEEE INFOCOM 2016, pp. 1–9, April 2016.

12. [12] V. Bajpai and J. Schonw ¨ alder, “IPv4 versus IPv6-Who connects faster?” in Proceedings of the 2015 14th IFIP Networking Conference, IFIP Networking 2015, pp. 1–9, May 2015.

13. M. Laor and L. Gendel, “The effect of packet reordering in a backbone link on application throughput,” IEEE Network, vol. 16, no. 5, pp. 28–36, 2002.

14. Y. Wang, G. Lu, and X. Li, “A study of Internet packet reordering,” in Proceedings of the International Conference on Information Networking, Lecture Notes in Computer Science, pp. 350–359, 2004.

15. X. Zhou and P. Van Mieghem, “Reordering of IP packets in Internet,” in Proceedings of the Passive and Active Network Measurement, pp. 237–246, 2004.

16. S. Bohacek, J. P. Hespanha, J. Lee, C. Lim, and K. Obraczka, “TCP-PR: TCP for persistent packet reordering,” in Proceedings of the 23th IEEE International Conference on Distributed Computing Systems, pp. 222–231, May 2003.

17. M. Przybylski, B. Belter, and A. Binczewski, “Shall we worry about packet reordering?” Computational Methods in Science and Technology, vol. 11, no. 2, pp. 141–146, 2005.

18. S. Dharmapurikar and V. Paxson, “Robust TCP Stream Reassembly in the Presence of Adversaries,” in Proceedings of the USENIX Security Symposium, 2005.

19. G. Varghese, J. Andrew Fingerhut, and F. Bonomi, “Detecting evasion attacks at high speeds without reassembly,” ACM SIGCOMM Computer Communication Review, vol. 36, no. 4, pp. 327–338, 2006.

20. M. Vutukuru, H. Balakrishnan, and V. Paxson, “Efficient and robust TCP stream normalization,” Security and Privacy, pp. 96–110, 2008.

21. Cisco Systems, IPv6 Performance Testing: A Comparison of IPv4 and IPv6 Forwarding Performance, Cisco White Paper, Government Systems Engineering, 2009.

22. R. Gilligan and E. Nordmark, “Transition Mechanisms for IPv6 Hosts and Routers,” RFC 7059, IETF, 2013.

23. Cisco Systems, IPv6 Deployment Guide, Cisco Documentation, 2018.

24. J. McCann, S. Deering, and J. Mogul, “Path MTU Discovery for IP Version 6,” RFC 8201, IETF, 2017.

25. APNIC Training, IPv6 Fundamentals and Deployment, APNIC, 2016.